

MODULE 1**Introduction to Java**

The Java Language, Java Development Kit (JDK); Java Buzzwords, Byte Code, JVM, JRE and Java environment, Data types, variables and Arrays, Operators, Control statement, command line Arguments, Object Oriented concepts, Classes, Objects and Methods, Access specifiers, Method Overloading, Constructor, Implicit this.

1.1 Java Programming Language

Here are important landmarks from the history of the Java language:

- The Java language was initially called OAK.
- Originally, it was developed for handling portable devices and set-top boxes. Oak was a massive failure.
- In 1995, Sun changed the name to "Java" and modified the language to take advantage of the burgeoning www (World Wide Web) development business.
- Later, in 2009, Oracle Corporation acquired Sun Microsystems and took ownership of three key Sun software assets: Java, MySQL, and Solaris.

1.2 Components of Java Programming Language**Java Development kit (JDK)**

JDK is a software development environment used for making applets and Java applications. The full form of JDK is Java Development Kit. Java developers can use it on Windows, macOS, Solaris, and Linux. JDK helps them to code and run Java programs. It is possible to install more than one JDK version on the same computer.

Why use JDK?

Here are the main reasons for using JDK:

- JDK contains tools required to write Java programs and JRE to execute them.
- It includes a compiler, Java application launcher, Applet viewer, etc.
- Compiler converts code written in Java into byte code.
- Java application launcher opens a JRE, loads the necessary class, and executes its main method.

Java Virtual Machine (JVM):

Java Virtual Machine (JVM) is an engine that provides a runtime environment to drive the Java Code or applications. It converts Java bytecode into machine language. JVM is a part of the Java Run Environment (JRE). In other programming languages, the compiler produces machine code for a particular system. However, the Java compiler produces code for a Virtual Machine known as Java Virtual Machine.

Why use JVM?

Here are the important reasons of using JVM:

- JVM provides a platform-independent way of executing Java source code.
- It has numerous libraries, tools, and frameworks.
- Once you run a Java program, you can run on any platform and save lots of time.
- JVM comes with JIT (Just-in-Time) compiler that converts Java source code into low-level machine language. Hence, it runs faster than a regular application.

Java Runtime Environment (JRE)

JRE is a piece of software that is designed to run other software. It contains the class libraries, loader class, and JVM. In simple terms, if you want to run a Java program, you need JRE. If you are not a programmer, you don't need to install JDK, but just JRE to run Java programs.

Why use JRE?

Here are the main reasons of using JRE:

- JRE contains class libraries, JVM, and other supporting files. It does not include any tool for Java development like a debugger, compiler, etc.
- It uses important package classes like math, swing, util, lang, awt, and runtime libraries.
- If you have to run Java applets, then JRE must be installed in your system.

Different Types of Java Platforms

There are four different types of Java programming language platforms:

1. Java Platform, Standard Edition (Java SE): Java SE's API offers the Java programming language's core functionality. It defines all the basis of type and object to high-level classes. It is used for networking, security, database access, graphical user interface (GUI) development, and XML parsing.

2. Java Platform, Enterprise Edition (Java EE): The Java EE platform offers an API and runtime environment for developing and running highly scalable, large-scale, multi-tiered, reliable, and secure network applications.

3. Java Programming Language Platform, Micro Edition (Java ME): The Java ME platform offers an API and a small-footprint virtual machine running Java programming language applications on small devices, like mobile phones.

4. Java FX: JavaFX is a platform for developing rich internet applications using a lightweight user-interface API. It uses hardware-accelerated graphics and media engines that help Java take advantage of higher-performance clients and a modern look-and-feel and high-level APIs for connecting to networked data sources.

1.3 Features of Java- Java Buzzwords

The primary objective of [Java programming](#) language creation was to make it portable, simple and secure programming language. Apart from this, there are also some excellent features which play an important role in the popularity of this language. The features of Java are also known as java *buzzwords*.

A list of most important features of Java language is given below.

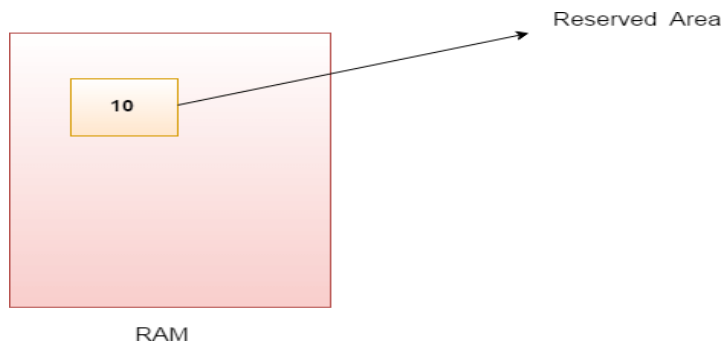
1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High Performance
10. Multithreaded
11. Distributed
12. Dynamic

1.4 Java Variables

A variable is a container which holds the value while the [Java program](#) is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: local, instance and static.

Variable is name of *reserved area allocated in memory*. In other words, it is a *name of memory location*. It is a combination of "vary + able" that means its value can be changed.



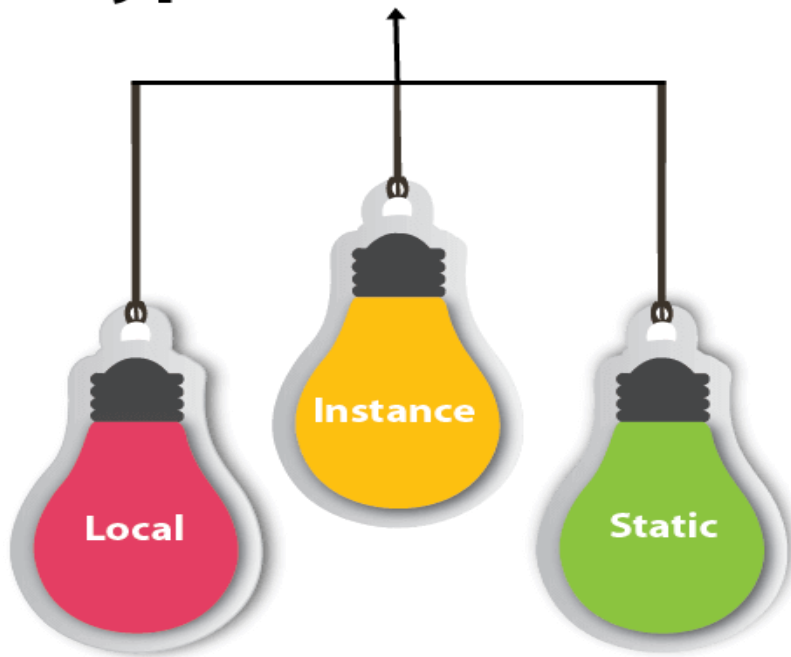
`int data=50;`//Here data is variable

Types of Variables

There are three types of variables in [Java](#):

- local variable
- instance variable
- static variable

Types of Variables



1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called instance variable. It is not declared as [static](#). It is called instance variable because its value is instance specific and is not shared among instances.

3) Static variable

A variable which is declared as static is called static variable. It cannot be local. You can create a single copy of static variable and share among all the instances of the class. Memory allocation for static variable happens only once when the class is loaded in the memory.

Example to understand the types of variables in java

```
class A{
    int data=50;//instance variable
    static int m=100;//static variable
    void method(){
        int n=90;//local variable
    }
} //end of class
```

Java Variable Example: Add Two Numbers

```
class Simple{
    public static void main(String[] args){
        int a=10;
        int b=10;
        int c=a+b;
        System.out.println(c);
    }
}
```

Output:

20

Java Variable Example: Widening

1. **class** Simple{
2. **public static void** main(String[] args){
3. **int** a=10;
4. **float** f=a;
5. System.out.println(a);

```
6.      System.out.println(f);
7.      }}
```

Output:

```
10
10.0
```

Java Variable Example: Narrowing (Typecasting)

```
1.      class Simple{
2.      public static void main(String[] args){
3.      float f=10.5f;
4.      //int a=f;//Compile time error
5.      int a=(int)f;
6.      System.out.println(f);
7.      System.out.println(a);
8.      }}
```

Output:

```
10.5
10
```

Java Variable Example: Overflow

```
1.      class Simple{
2.      public static void main(String[] args){
3.      //Overflow
4.      int a=130;
5.      byte b=(byte)a;
6.      System.out.println(a);
7.      System.out.println(b);
8.      }}
```

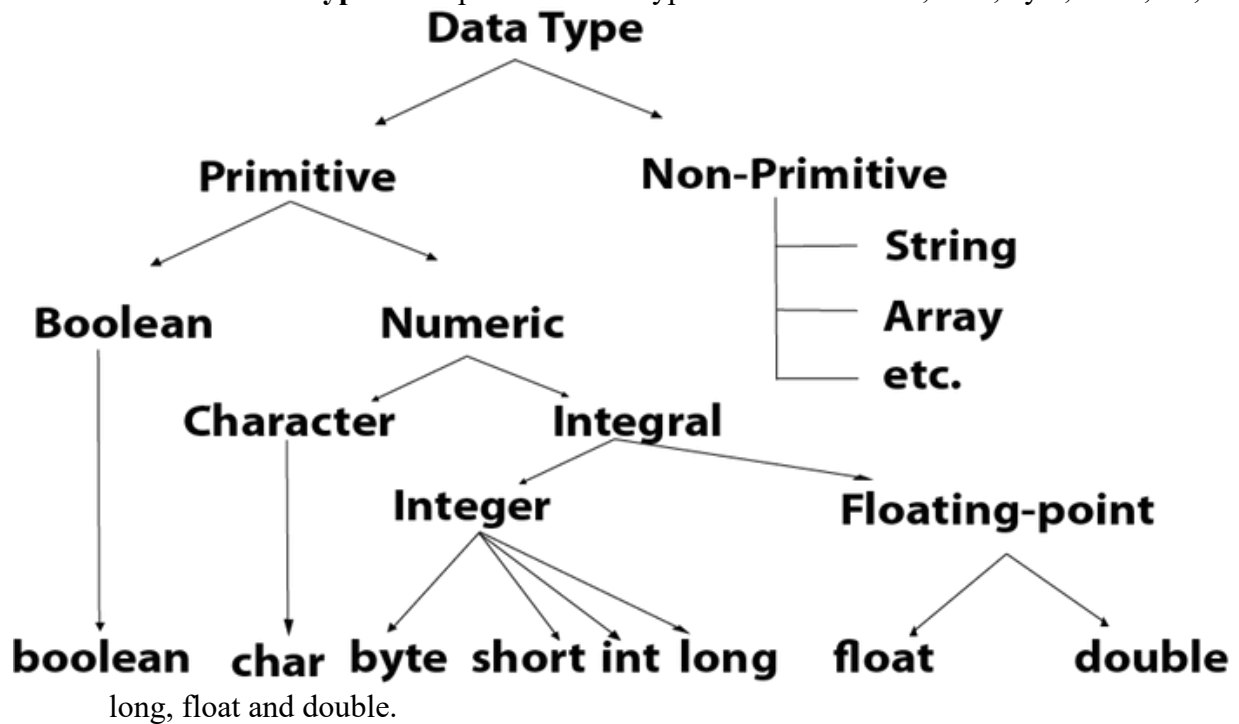
Output:

```
130
-126
```

1.5 Data Types in Java

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int,



2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.

Java Primitive Data Types

In Java language, primitive data types are the building blocks of data manipulation. These are the most basic data types available in [Java language](#).

Java is a statically-typed programming language. It means, all [variables](#) must be declared before its use. That is why we need to declare variable's type and name.

There are 8 types of primitive data types:

- boolean data type
- byte data type
- char data type
- short data type
- int data type
- long data type
- float data type
- double data type

TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS	RANGE OF VALUES
boolean	true or false	false	1 bit	true, false	true, false
byte	twos complement integer	0	8 bits	(none)	-128 to 127
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\', '\n', '\b'	character representation of ASCII values 0 to 255
short	twos complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2	-2,147,483,648 to 2,147,483,647
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F	upto 7 decimal digits
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d	upto 16 decimal digits

Boolean Data Type

The Boolean data type is used to store only two possible values: true and false. This data type is used for simple flags that track true/false conditions.

The Boolean data type specifies one bit of information, but its "size" can't be defined precisely.

Example: Boolean one = false

Byte Data Type

The byte data type is an example of primitive data type. It is an 8-bit signed two's complement integer. Its value-range lies between -128 to 127 (inclusive). Its minimum value is -128 and maximum value is 127. Its default value is 0.

The byte data type is used to save memory in large arrays where the memory savings is most required. It saves space because a byte is 4 times smaller than an integer. It can also be used in place of "int" data type.

Example: byte a = 10, byte b = -20

Short Data Type

The short data type is a 16-bit signed two's complement integer. Its value-range lies between -32,768 to 32,767 (inclusive). Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0.

The short data type can also be used to save memory just like byte data type. A short data type is 2 times smaller than an integer.

Example: short s = 10000, short r = -5000

Int Data Type

The int data type is a 32-bit signed two's complement integer. Its value-range lies between -2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31} - 1$) (inclusive). Its minimum value is -2,147,483,648 and maximum value is 2,147,483,647. Its default value is 0.

The int data type is generally used as a default data type for integral values unless if there is no problem about memory.

Example: int a = 100000, int b = -200000

Long Data Type

The long data type is a 64-bit two's complement integer. Its value-range lies between -9,223,372,036,854,775,808 (-2^{63}) to 9,223,372,036,854,775,807 ($2^{63} - 1$) (inclusive). Its minimum value is -9,223,372,036,854,775,808 and maximum value is 9,223,372,036,854,775,807. Its default value is 0. The long data type is used when you need a range of values more than those provided by int.

Example: long a = 100000L, long b = -200000L

Float Data Type

The float data type is a single-precision 32-bit IEEE 754 floating point. Its value range is unlimited. It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating-point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Example: float f1 = 234.5f

Double Data Type

The double data type is a double-precision 64-bit IEEE 754 floating point. Its value range is unlimited. The double data type is generally used for decimal values just like float. The double data type also should never be used for precise values, such as currency. Its default value is 0.0d.

Example: double d1 = 12.3

Char Data Type

The char data type is a single 16-bit Unicode character. Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive). The char data type is used to store characters.

Example: char letterA = 'A'

Why char uses 2 byte in java and what is '\u0000' ?

It is because java uses Unicode system not ASCII code system. The '\u0000' is the lowest range of Unicode system.

Java program to demonstrate
// primitive data types in Java

```
classGeeksforGeeks {  
    publicstaticvoidmain(String args[])  
    {  
        // declaring character  
        chara = 'G';  
    }  
}
```

```
// Integer data type is generally
// used for numeric values
inti = 89;

// use byte and short
// if memory is a constraint
byteb = 4;

// this will give error as number is
// larger than byte range
// byte b1 = 78888888955;

shorts = 56;

// this will give error as number is
// larger than short range
// short s1 = 87878787878;

// by default fraction value
// is double in java
doubled = 4.355453532;

// for float use 'f' as suffix
floatf = 4.7333434f;

System.out.println("char: " + a);
System.out.println("integer: " + i);
System.out.println("byte: " + b);
System.out.println("short: " + s);
System.out.println("float: " + f);
System.out.println("double: " + d);
}
```

Output:

```
char: G
integer: 89
byte: 4
short: 56
float: 4.7333436
double: 4.355453532
```

1.6 JAVA ARRAYS

Java provides a data structure, the **array**, which stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more

useful to think of an array as a collection of variables of the same type.

Instead of declaring individual variables, such as `number0`, `number1`, ..., and `number99`, you declare one array variable such as `numbers` and use `numbers[0]`, `numbers[1]`, and ..., `numbers[99]` to represent individual variables.

This tutorial introduces how to declare array variables, create arrays, and process arrays using indexed variables.

Declaring Array Variables

To use an array in a program, you must declare a variable to reference the array, and you must specify the type of array the variable can reference. Here is the syntax for declaring an array variable –

Syntax

```
dataType[] arrayRefVar; // preferred way.
```

or

```
dataType arrayRefVar[]; // works but not preferred way.
```

Note – The style **`dataType[] arrayRefVar`** is preferred. The style **`dataType arrayRefVar[]`** comes from the C/C++ language and was adopted in Java to accommodate C/C++ programmers.

Example

The following code snippets are examples of this syntax –

```
double[] myList; // preferred way.  
or  
double myList[]; // works but not preferred way.
```

Creating Arrays

You can create an array by using the `new` operator with the following syntax –

Syntax

```
arrayRefVar = new dataType[arraySize];
```

The above statement does two things –

- It creates an array using `new dataType[arraySize]`.
- It assigns the reference of the newly created array to the variable `arrayRefVar`.

Declaring an array variable, creating an array, and assigning the reference of the array to the variable can be combined in one statement, as shown below –

```
dataType[] arrayRefVar = new dataType[arraySize];
```

Alternatively you can create arrays as follows –

```
dataType[] arrayRefVar = {value0, value1, ..., valuek};
```

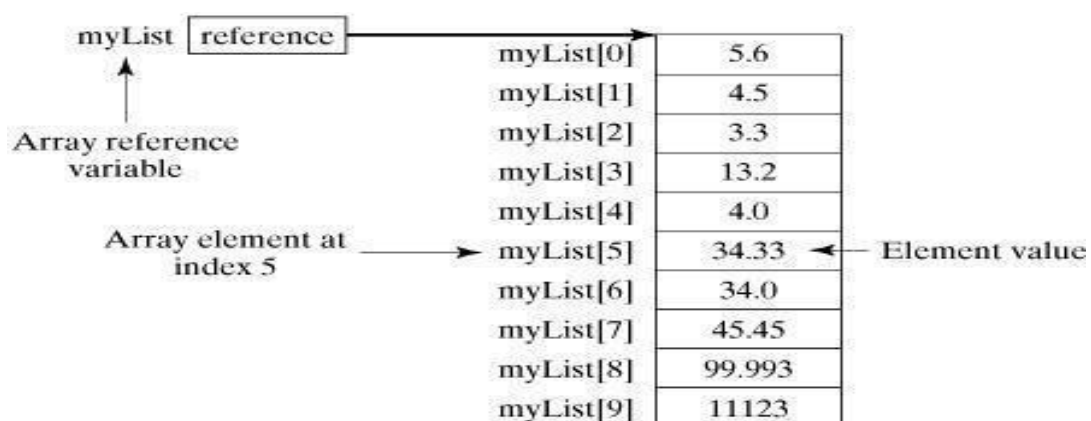
The array elements are accessed through the **index**. Array indices are 0-based; that is, they start from 0 to **arrayRefVar.length-1**.

Example

Following statement declares an array variable, `myList`, creates an array of 10 elements of double type and assigns its reference to `myList` –

```
double[] myList = new double[10];
```

Following picture represents array `myList`. Here, `myList` holds ten double values and the indices are from 0 to 9.



Processing Arrays

When processing array elements, we often use either **for** loop or **foreach** loop because all of the elements in an array are of the same type and the size of the array is known.

Example

Here is a complete example showing how to create, initialize, and process arrays –

```
public class TestArray {  
  
    public static void main(String[] args) {  
        double[] myList = {1.9, 2.9, 3.4, 3.5};  
  
        // Print all the array elements  
        for (int i = 0; i < myList.length; i++) {  
            System.out.println(myList[i] + " ");  
        }  
  
        // Summing all elements  
        double total = 0;  
        for (int i = 0; i < myList.length; i++) {  
            total += myList[i];  
        }  
        System.out.println("Total is " + total);  
  
        // Finding the largest element  
        double max = myList[0];  
        for (int i = 1; i < myList.length; i++) {  
            if (myList[i] > max) max = myList[i];  
        }  
        System.out.println("Max is " + max);  
    }  
}
```

This will produce the following result –

Output

```
1.9  
2.9  
3.4  
3.5  
Total is 11.7  
Max is 3.5
```

1.7 Types of Operators in Java

- 1) Basic Arithmetic Operators
- 2) Assignment Operators
- 3) Auto-increment and Auto-decrement Operators
- 4) Logical Operators
- 5) Comparison (relational) operators
- 6) Bitwise Operators
- 7) Ternary Operator

1) Basic Arithmetic Operators

Basic arithmetic operators are: +, -, *, /, %

+ is for addition.

– is for subtraction.

* is for multiplication.

/ is for division.

% is for modulo.

Note: Modulo operator returns remainder, for example 10 % 5 would return 0

Example of Arithmetic Operators

```
public class ArithmeticOperatorDemo {  
    public static void main(String args[]) {  
        int num1 = 100;  
        int num2 = 20;  
  
        System.out.println("num1 + num2: " + (num1 + num2) );  
        System.out.println("num1 - num2: " + (num1 - num2) );  
        System.out.println("num1 * num2: " + (num1 * num2) );  
    }  
}
```



```
    System.out.println("num1 / num2: " + (num1 / num2) );  
    System.out.println("num1 % num2: " + (num1 % num2) );  
}  
}
```

Output:

```
num1 + num2: 120  
num1 - num2: 80  
num1 * num2: 2000  
num1 / num2: 5  
num1 % num2: 0
```

Checkout these [java programs](#) related to arithmetic Operators in Java:

1. [Java Program to Add two numbers](#)
2. [Java Program to Multiply two Numbers](#)

2) Assignment Operators

Assignments operators in java are: =, +=, -=, *=, /=, %=

num2 = num1 would assign value of variable num1 to the variable.

num2+=num1 is equal to num2 = num2+num1

num2-=num1 is equal to num2 = num2-num1

num2*=num1 is equal to num2 = num2*num1

num2/=num1 is equal to num2 = num2/num1

num2%=num1 is equal to num2 = num2%num1

Example of Assignment Operators

```
public class AssignmentOperatorDemo {  
    public static void main(String args[]) {  
        int num1 = 10;  
        int num2 = 20;  
  
        num2 = num1;  
        System.out.println("= Output: "+num2);  
  
        num2 += num1;  
        System.out.println("+= Output: "+num2);  
  
        num2 -= num1;  
        System.out.println("-= Output: "+num2);  
    }  
}
```

```
num2 *= num1;  
System.out.println("*= Output: "+num2);  
  
num2 /= num1;  
System.out.println("/= Output: "+num2);  
  
num2 %= num1;  
System.out.println("%= Output: "+num2);  
}  
}
```

Output:

```
= Output: 10  
+= Output: 20  
-= Output: 10  
*= Output: 100  
/= Output: 10  
%= Output: 0
```

3) Auto-increment and Auto-decrement Operators

++ and --

num++ is equivalent to num=num+1;

num-- is equivalent to num=num-1;

Example of Auto-increment and Auto-decrement Operators

```
public class AutoOperatorDemo {  
    public static void main(String args[]){  
        int num1=100;  
        int num2=200;  
        num1++;  
        num2--;  
        System.out.println("num1++ is: "+num1);  
        System.out.println("num2-- is: "+num2);  
    }  
}
```

Output:

```
num1++ is: 101  
num2-- is: 199
```

4) Logical Operators

Logical Operators are used with binary variables. They are mainly used in conditional statements and loops for evaluating a condition.

Logical operators in java are: &&, ||, !

Let's say we have two boolean variables b1 and b2.

b1&&b2 will return true if both b1 and b2 are true else it would return false.

b1||b2 will return false if both b1 and b2 are false else it would return true.

!b1 would return the opposite of b1, that means it would be true if b1 is false and it would return false if b1 is true.

Example of Logical Operators

```
public class LogicalOperatorDemo {  
    public static void main(String args[]) {  
        boolean b1 = true;  
        boolean b2 = false;  
  
        System.out.println("b1 && b2: " + (b1&&b2));  
        System.out.println("b1 || b2: " + (b1||b2));  
        System.out.println("!(b1 && b2): " + !(b1&&b2));  
    }  
}
```

Output:

```
b1 && b2: false  
b1 || b2: true  
!(b1 && b2): true
```

5) Comparison(Relational) operators

We have six relational operators in Java: ==, !=, >, <, >=, <=

== returns true if both the left side and right side are equal

!= returns true if left side is not equal to the right side of operator.

> returns true if left side is greater than right.

< returns true if left side is less than right side.

>= returns true if left side is greater than or equal to right side.

<= returns true if left side is less than or equal to right side.

Example of Relational operators

Note: This example is using if-else statement which is our next tutorial, if you are finding it difficult to understand then refer [if-else in Java](#).

```
public class RelationalOperatorDemo {
    public static void main(String args[]) {
        int num1 = 10;
        int num2 = 50;
        if (num1==num2) {
            System.out.println("num1 and num2 are equal");
        }
        else{
            System.out.println("num1 and num2 are not equal");
        }

        if( num1 != num2 ){
            System.out.println("num1 and num2 are not equal");
        }
        else{
            System.out.println("num1 and num2 are equal");
        }

        if( num1 > num2 ){
            System.out.println("num1 is greater than num2");
        }
        else{
            System.out.println("num1 is not greater than num2");
        }

        if( num1 >= num2 ){
            System.out.println("num1 is greater than or equal to num2");
        }
        else{
            System.out.println("num1 is less than num2");
        }

        if( num1 < num2 ){
            System.out.println("num1 is less than num2");
        }
        else{
            System.out.println("num1 is not less than num2");
        }
    }
}
```

```

if( num1 <= num2){
    System.out.println("num1 is less than or equal to num2");
}
else{
    System.out.println("num1 is greater than num2");
}
}
}

```

6) Bitwise Operators

There are six bitwise Operators: **&**, **|**, **^**, **~**, **<<**, **>>**

```

num1 = 11; /* equal to 00001011*/
num2 = 22; /* equal to 00010110 */

```

Bitwise operator performs bit by bit processing.

num1 & num2 compares corresponding bits of num1 and num2 and generates 1 if both bits are equal, else it returns 0. In our case it would return: 2 which is 00000010 because in the binary form of num1 and num2 only second last bits are matching.

num1 | num2 compares corresponding bits of num1 and num2 and generates 1 if either bit is 1, else it returns 0. In our case it would return 31 which is 00011111

num1 ^ num2 compares corresponding bits of num1 and num2 and generates 1 if they are not equal, else it returns 0. In our example it would return 29 which is equivalent to 00011101

~num1 is a complement operator that just changes the bit from 0 to 1 and 1 to 0. In our example it would return -12 which is signed 8 bit equivalent to 11110100

num1 << 2 is left shift operator that moves the bits to the left, discards the far left bit, and assigns the rightmost bit a value of 0. In our case output is 44 which is equivalent to 00101100

Note: In the example below we are providing 2 at the right side of this shift operator that is the reason bits are moving two places to the left side. We can change this number and bits would be moved by the number of bits specified on the right side of the operator. Same applies to the right side operator.

num1 >> 2 is right shift operator that moves the bits to the right, discards the far right bit, and assigns the leftmost bit a value of 0. In our case output is 2 which is equivalent to 00000010

Example of Bitwise Operators

```
public class BitwiseOperatorDemo {
    public static void main(String args[]) {

        int num1 = 11; /* 11 = 00001011 */
        int num2 = 22; /* 22 = 00010110 */
        int result = 0;

        result = num1 & num2;
        System.out.println("num1 & num2: "+result);

        result = num1 | num2;
        System.out.println("num1 | num2: "+result);

        result = num1 ^ num2;
        System.out.println("num1 ^ num2: "+result);

        result = ~num1;
        System.out.println("~num1: "+result);

        result = num1 << 2;
        System.out.println("num1 << 2: "+result); result = num1 >> 2;
        System.out.println("num1 >> 2: "+result);
    }
}
```

Output:

```
num1 & num2: 2
num1 | num2: 31
num1 ^ num2: 29
~num1: -12
num1 << 2: 44 num1 >> 2: 2
```

Check out this program: [Java Program to swap two numbers using bitwise operator](#)

7) Ternary Operator

This operator evaluates a boolean expression and assign the value based on the result.

```
public class TernaryOperatorDemo {  
  
    public static void main(String args[]) {  
        int num1, num2;  
        num1 = 25;  
        /* num1 is not equal to 10 that's why  
         * the second value after colon is assigned  
         * to the variable num2  
         */  
        num2 = (num1 == 10) ? 100: 200;  
        System.out.println( "num2: "+num2);  
  
        /* num1 is equal to 25 that's why  
         * the first value is assigned  
         * to the variable num2  
         */  
        num2 = (num1 == 25) ? 100: 200;  
        System.out.println( "num2: "+num2);  
    }  
}
```

Output:

```
num2: 200  
num2: 100
```

1.8 Control Statement- Decision Making in Java (if, if-else, switch, break, continue, jump)

Decision Making in programming is similar to decision making in real life. In programming, also we face some situations where we want a certain block of code to be executed when some condition is fulfilled. A programming language uses control statements to control the flow of execution of program based on certain conditions. These are used to cause the flow of execution to advance and branch based on changes to the state of a program.

Java's Selection statements:

if

if-else

nested-if

if-else-if

switch-case

jump – break, continue, return

These statements allow you to control the flow of your program's execution based upon conditions known only during run time.

if:

if statement is the simplest decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statement is executed otherwise not.

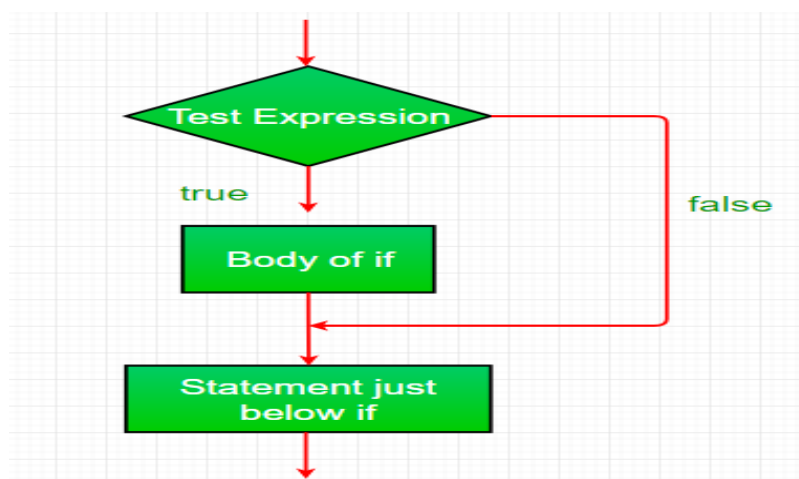
Syntax:

```
if(condition)
{
    // Statements to execute if
    // condition is true
}
```

Here, **condition** after evaluation will be either true or false. if statement accepts boolean values – if the value is true then it will execute the block of statements under it. If we do not provide the curly braces '{' and '}' after **if(condition)** then by default if statement will consider the immediate one statement to be inside its block. For example

```
if(condition)
    statement1;
    statement2;

// Here if the condition is true, if block
// will consider only statement1 to be inside
// its block.
```



Flow chart:

Example:

```
// Java program to illustrate If statement
class IfDemo
{
    public static void main(String args[])
    {
        int i = 10;

        if (i > 15)
            System.out.println("10 is less than 15");

        // This statement will be executed
        // as if considers one statement by default
        System.out.println("I am Not in if");
    }
}
```

Output:

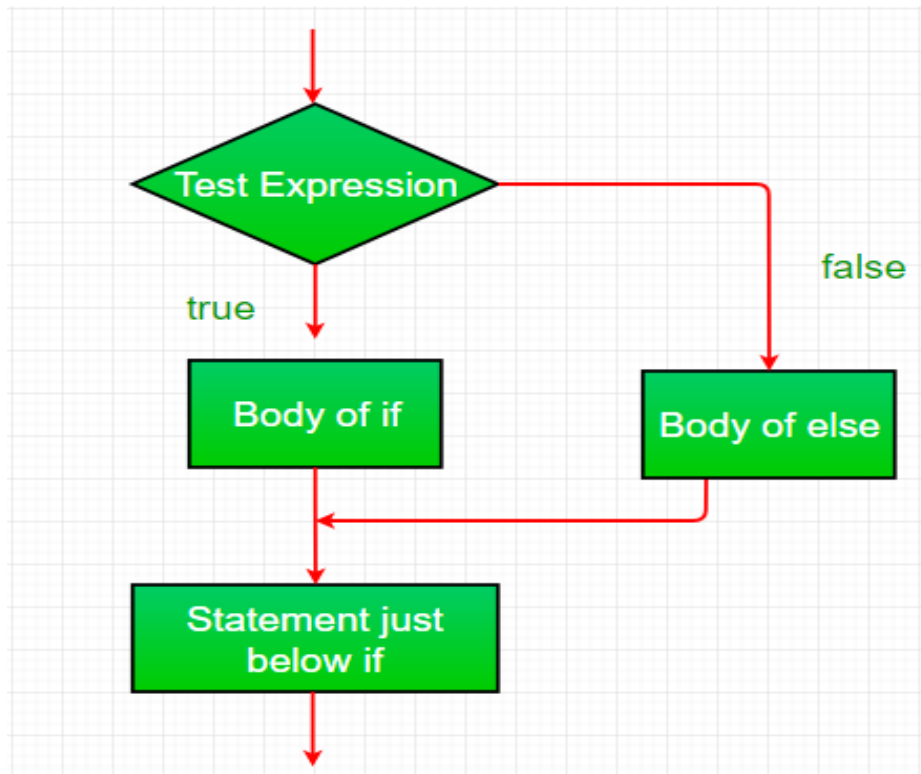
```
I am Not in if
```

if-else

The if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else if the condition is false. Here comes the else statement. We can use the else statement with if statement to execute a block of code when the condition is false.

Syntax:

```
if(condition)
{
    // Executes this block if
    // condition is true
}
else
{
    // Executes this block if
    // condition is false
}
```



Example:

```
// Java program to illustrate if-else statement
class IfElseDemo
{
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)
            System.out.println("i is smaller than 15");
        else
            System.out.println("i is greater than 15");
    }
}
```

Output:

i is smaller than 15

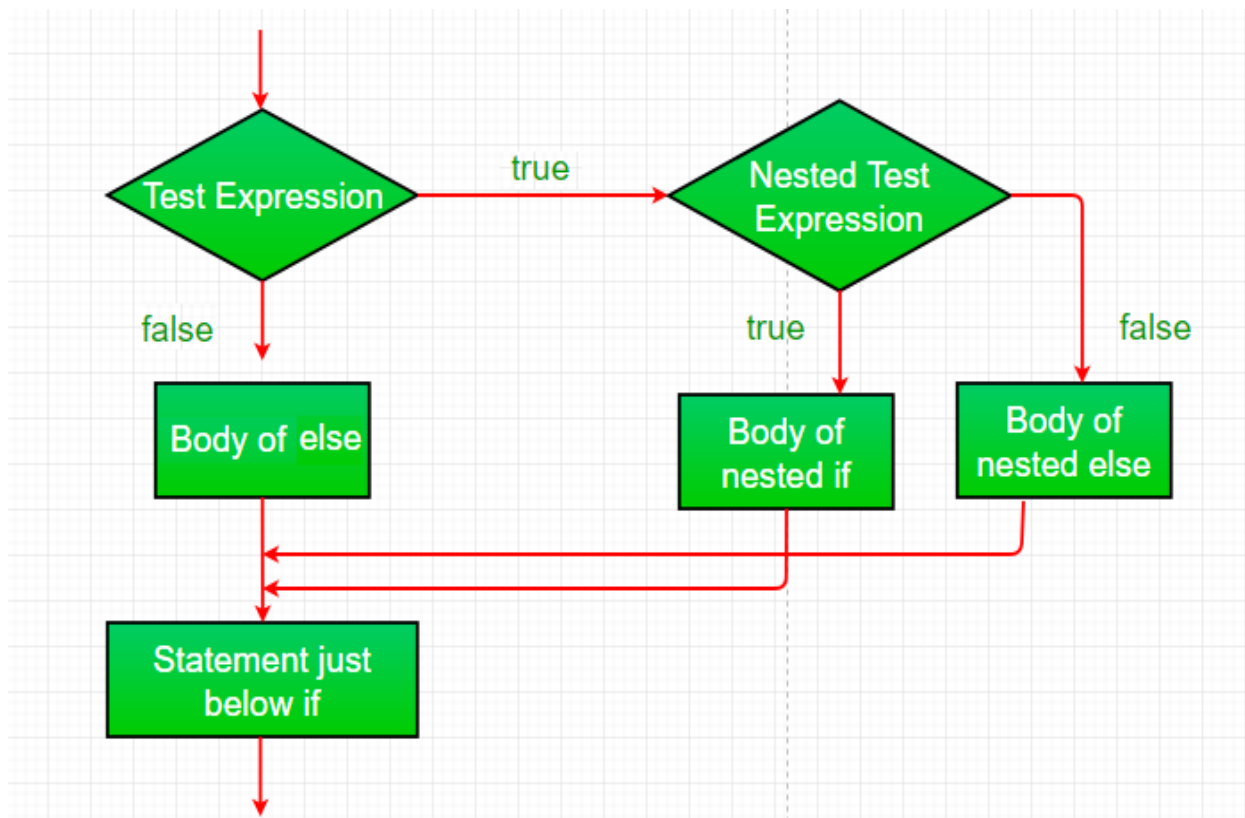
nested-if

A nested if is an if statement that is the target of another if or else. Nested if statements mean an if statement inside an if statement. Yes, java allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.

Syntax:

```
if (condition1)
{
    // Executes when condition1 is true
    if (condition2)
    {
        // Executes when condition2 is true
    }
}
```

Flowchart:



Example:

```
// Java program to illustrate nested-if statement
class NestedIfDemo
{
    public static void main(String args[])
    {
        int i = 10;

        if(i == 10)
        {
            // First if statement
            if (i < 15)
                System.out.println("i is smaller than 15");

            // Nested - if statement
            // Will only be executed if statement above
            // it is true
            if (i < 12)
                System.out.println("i is smaller than 12 too");
            else
                System.out.println("i is greater than 15");
        }
    }
}
```

Output:

```
i is smaller than 15
i is smaller than 12 too
```

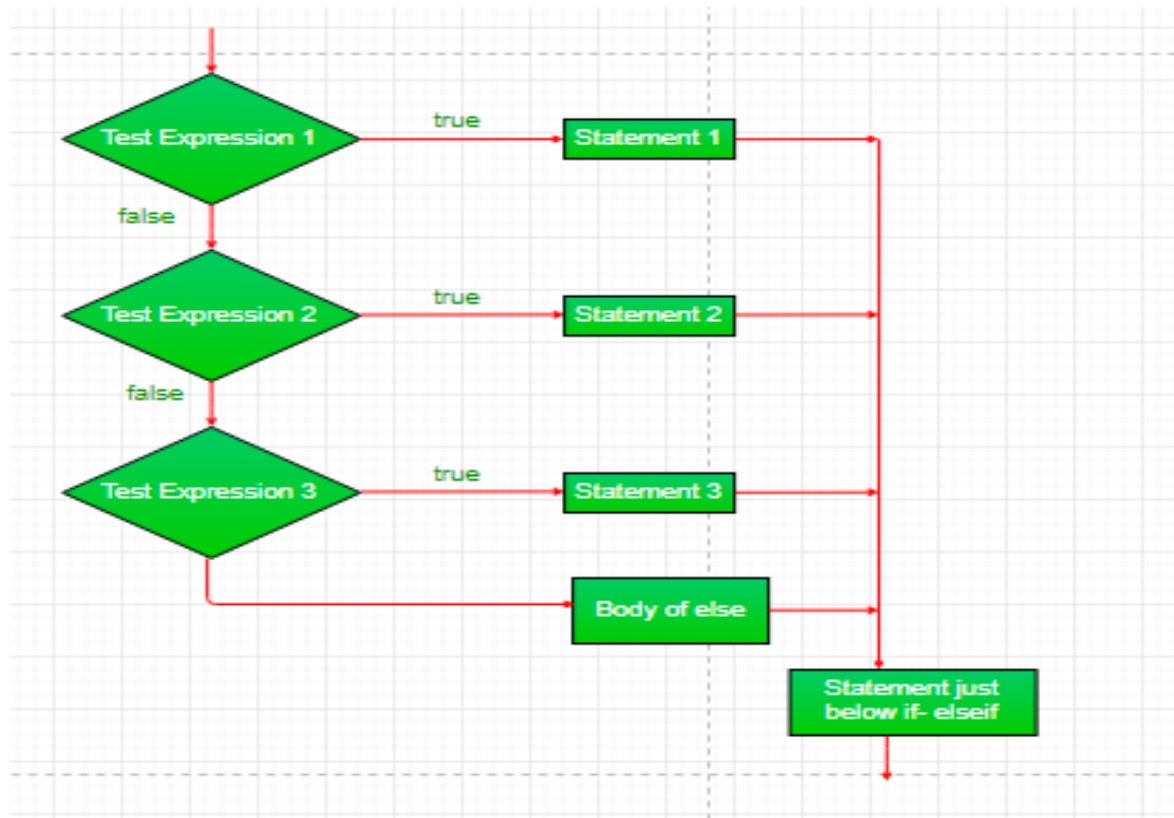
if-else-if ladder:

Here, a user can decide among multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.

Syntax:

```
if(condition)
    statement;
else if(condition)
    statement;
else
    statement;
```

Flowchart:



Example:

```
// Java program to illustrate if-else-if ladder
class ifelseifDemo
{
    public static void main(String args[])
    {
        int i = 20;

        if (i == 10)
            System.out.println("i is 10");
        else if (i == 15)
            System.out.println("i is 15");
        else if (i == 20)
            System.out.println("i is 20");
        else
```

```
        System.out.println("i is not present");  
    }  
}
```

Output:

```
i is 20
```

switch-case

The switch statement is a multiway branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression.

Syntax:

```
switch (expression)  
{  
    case value1:  
        statement1;  
        break;  
    case value2:  
        statement2;  
        break;  
    case valueN:  
        statementN;  
        break;  
    default:  
        statementDefault;  
}
```

Expression can be of type byte, short, int char or an enumeration. Beginning with JDK7, *expression* can also be of type String.

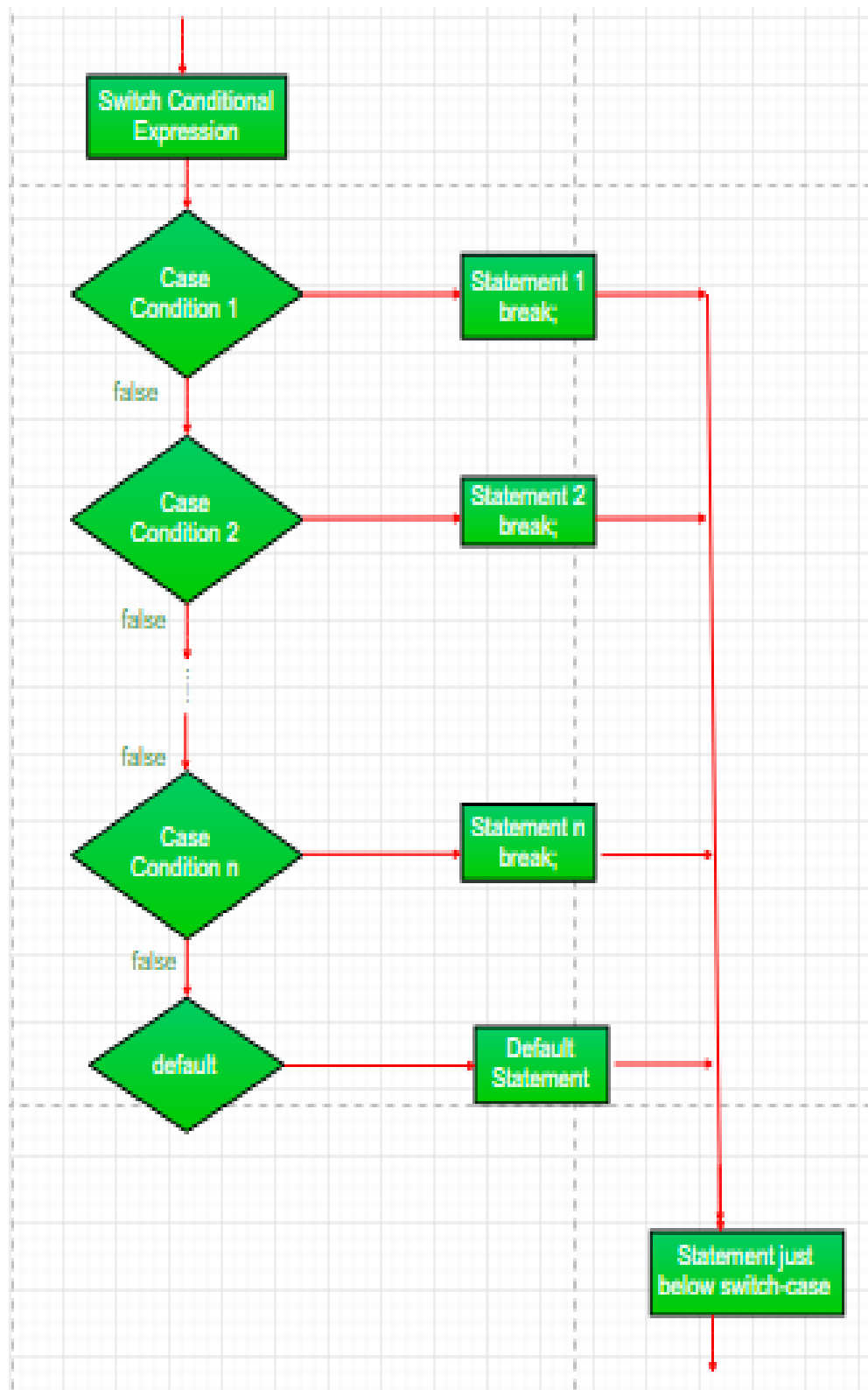
Duplicate case values are not allowed.

The default statement is optional.

The break statement is used inside the switch to terminate a statement sequence.

The break statement is optional. If omitted, execution will continue on into the next case.

Flowchart:



```
// Java program to illustrate switch-case
class SwitchCaseDemo
{
    public static void main(String args[])
    {
        int i = 9;
        switch (i)
        {
            case 0:
                System.out.println("i is zero.");
                break;
            case 1:
                System.out.println("i is one.");
                break;
            case 2:
                System.out.println("i is two.");
                break;
            default:
                System.out.println("i is greater than 2.");
        }
    }
}
```

Output:

i is greater than 2.

jump

Java supports three jump statement: break, continue and return. These three statements transfer control to other part of the program.

1Break:

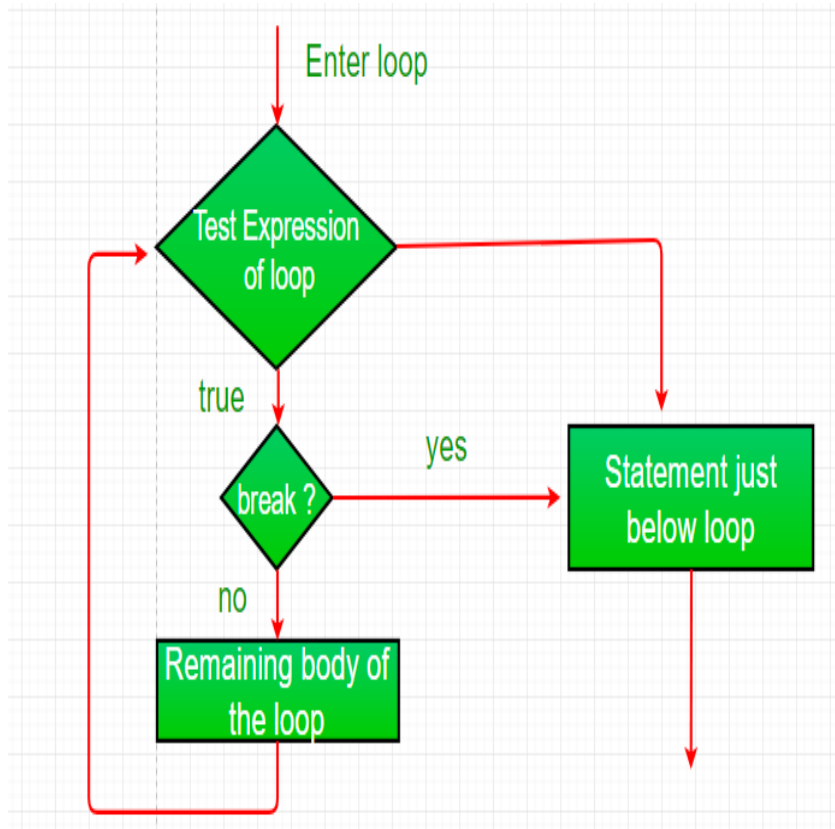
In Java, break is majorly used for:

- Terminate a sequence in a switch statement (discussed above). To exit a loop.
- Used as a “civilized” form of goto.
- Using break to exit a Loop

Using break, we can force immediate termination of a loop, bypassing the conditional expression and any remaining code in the body of the loop.

Note: Break, when used inside a set of nested loops, will only break out of the innermost loop.

Flowchart:



Example:

```
// Java program to illustrate using
// break to exit a loop
class BreakLoopDemo
{
    public static void main(String args[])
    {
        // Initially loop is set to run from 0-9
        for (int i = 0; i < 10; i++)
        {
            // terminate loop when i is 5.
            if (i == 5)
                break;

            System.out.println("i: " + i);
        }
        System.out.println("Loop complete.");
    }
}
```

Output:

```
i: 0
i: 1
i: 2
i: 3
```



```

        // second label
        if (t)
            break second;
        System.out.println("This won't execute.");
    }
    System.out.println("This won't execute.");
}

// First block
System.out.println("This is after second block.");
}
}
}
}
Output:

```

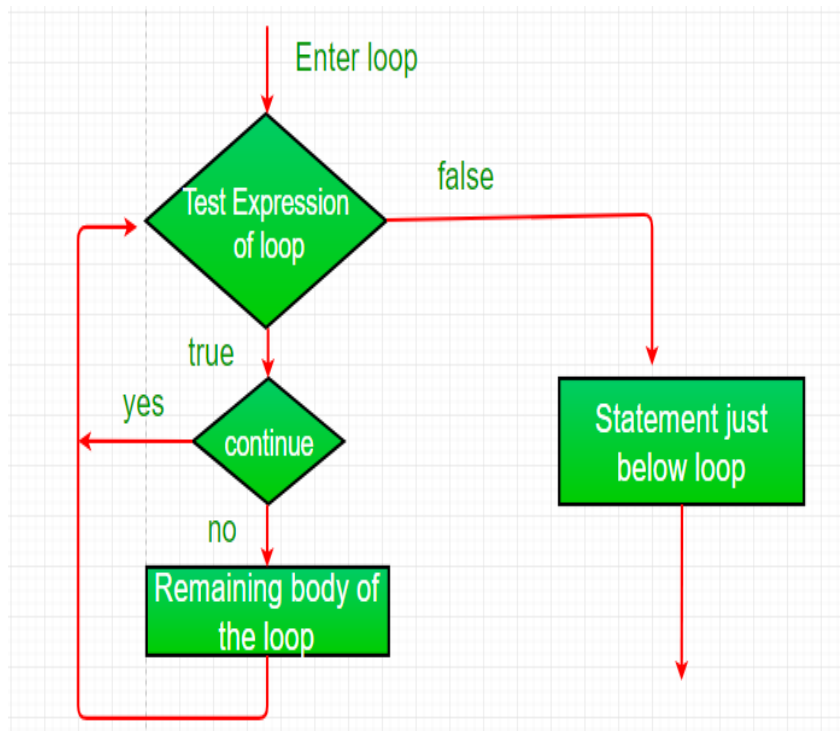
Before the break.

This is after second block.

2.Continue:

Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration. This is, in effect, a goto just past the body of the loop, to the loop's end. The continue statement performs such an action.

Flowchart:



Example:

```

// Java program to illustrate using
// continue in an if statement
class ContinueDemo
{

```

```
public static void main(String args[])
{
    for (int i = 0; i < 10; i++)
    {
        // If the number is even
        // skip and continue
        if (i%2 == 0)
            continue;

        // If number is odd, print it
        System.out.print(i + " ");
    }
}
```

Output:

```
1 3 5 7 9
```

3. **Return:** The return statement is used to explicitly return from a method. That is, it causes a program control to transfer back to the caller of the method.

Example:

```
filter_none
edit
play_arrow
brightness_4
```

// Java program to illustrate using return

```
class Return
{
    public static void main(String args[])
    {
        boolean t = true;
        System.out.println("Before the return.");

        if(t)
            return;

        // Compiler will bypass every statement
        // after return
        System.out.println("This won't execute.");
    }
}
```

Output:

```
Before the return.
```

1.9 Loops in Java

Looping in programming languages is a feature which facilitates the execution of a set of instructions/functions repeatedly while some condition evaluates to true.

Java provides three ways for executing the loops. While all the ways provide similar basic functionality, they differ in their syntax and condition checking time.

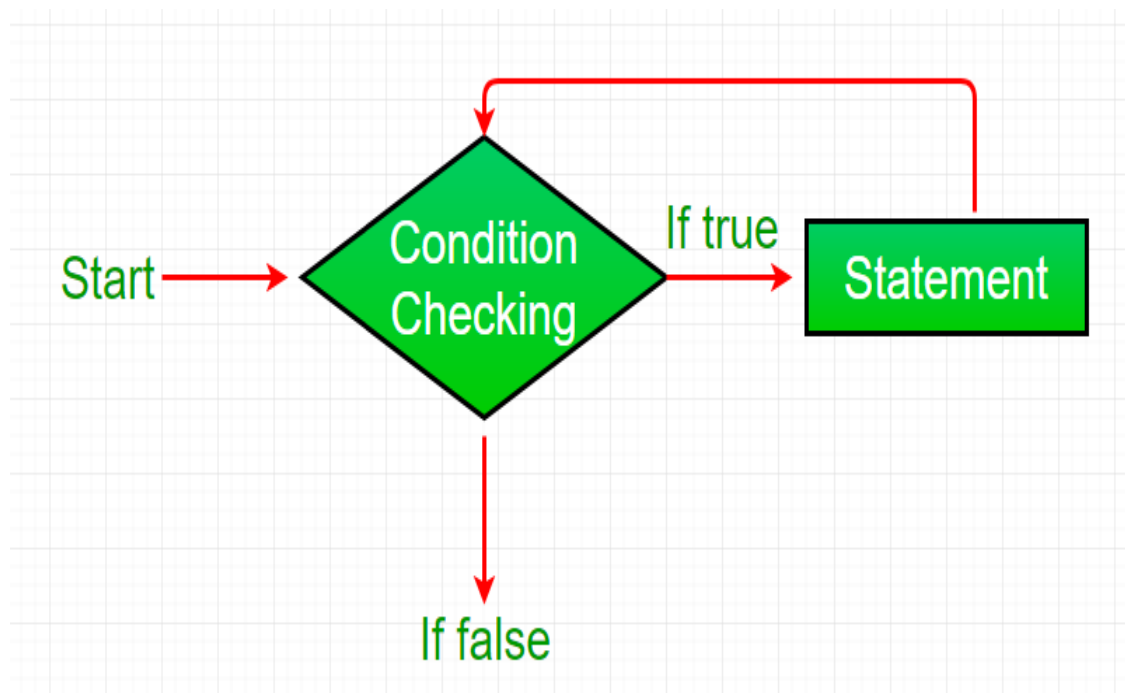
while loop:

A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.

Syntax:

```
while (boolean condition)  
{  
    loop statements...  
}
```

Flowchart:



- While loop starts with the checking of condition. If it evaluated to true, then the loop body statements are executed otherwise first statement following the loop is executed. For this reason, it is also called **Entry control loop**
- Once the condition is evaluated to true, the statements in the loop body are executed. Normally the statements contain an update value for the variable being processed for the next iteration.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.

```
// Java program to illustrate while loop
class whileLoopDemo
{
    public static void main(String args[])
    {
        int x = 1;

        // Exit when x becomes greater than 4
        while (x <= 4)
        {
            System.out.println("Value of x:" + x);

            // Increment the value of x for
            // next iteration
            x++;
        }
    }
}
```

Output:

Value of x:1

Value of x:2

Value of x:3

Value of x:4

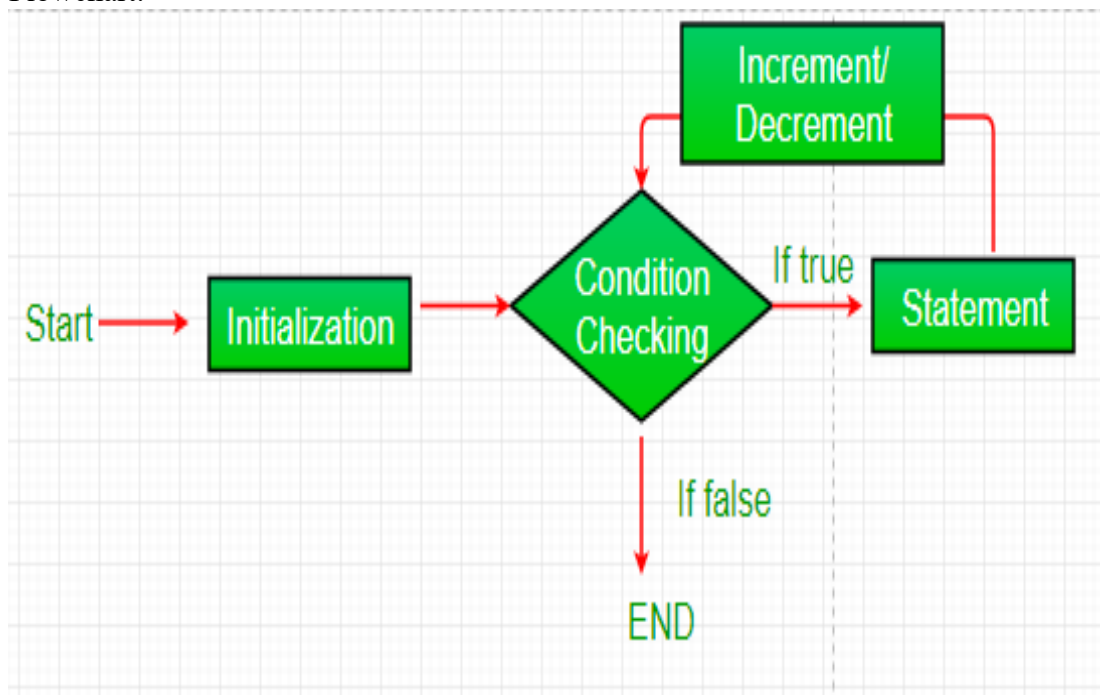
for loop:

for loop provides a concise way of writing the loop structure. Unlike a while loop, a for statement consumes the initialization, condition and increment/decrement in one line thereby providing a shorter, easy to debug structure of looping.

Syntax:

```
for (initialization condition; testing condition;  
    increment/decrement)  
{  
    statement(s)  
}
```

Flowchart:



1. **Initialization condition:** Here, we initialize the variable in use. It marks the start of a for loop. An already declared variable can be used or a variable can be declared, local to loop only.
2. **Testing Condition:** It is used for testing the exit condition for a loop. It must return a boolean value. It is also an **Entry Control Loop** as the condition is checked prior to the execution of the loop statements.
3. **Statement execution:** Once the condition is evaluated to true, the statements in the loop body are executed.
4. **Increment/ Decrement:** It is used for updating the variable for next iteration.
5. **Loop termination:** When the condition becomes false, the loop terminates marking the end of its life cycle.

```
// Java program to illustrate for loop.
class forLoopDemo
{
    public static void main(String args[])
    {
        // for loop begins when x=2
        // and runs till x <=4
        for (int x = 2; x <= 4; x++)
            System.out.println("Value of x:" + x);
    }
}
```

Output:

Value of x:2

Value of x:3

Value of x:4

Enhanced For loop

Java also includes another version of for loop introduced in Java 5. Enhanced for loop provides a simpler way to iterate through the elements of a collection or array. It is inflexible and should be used only when there is a need to iterate through the elements in sequential manner without knowing the index of currently processed element.

Also note that the object/variable is immutable when enhanced for loop is used i.e it ensures that the values in the array cannot be modified, so it can be said as read only loop where you can't update the values as opposite to other loops where values can be modified.

We recommend using this form of the for statement instead of the general form whenever possible. (as per JAVA doc.)

Syntax:


```
for (T element:Collection obj/array)
{
    statement(s)
}
```

Let's take an example to demonstrate how enhanced for loop can be used to simplify the work. Suppose there is an array of names and we want to print all the names in that array. Let's see the difference with these two examples

Enhanced for loop simplifies the work as follows-

```
// Java program to illustrate enhanced for loop
public class enhancedforloop
{
    public static void main(String args[])
    {
        String array[] = {"Ron", "Harry", "Hermoine"};

        //enhanced for loop
        for (String x:array)
        {
            System.out.println(x);
        }

        /* for loop for same function
        for (int i = 0; i < array.length; i++)
        {
            System.out.println(array[i]);
        }
        */
    }
}
```

Output:

```
Ron
Harry
Hermoine
```

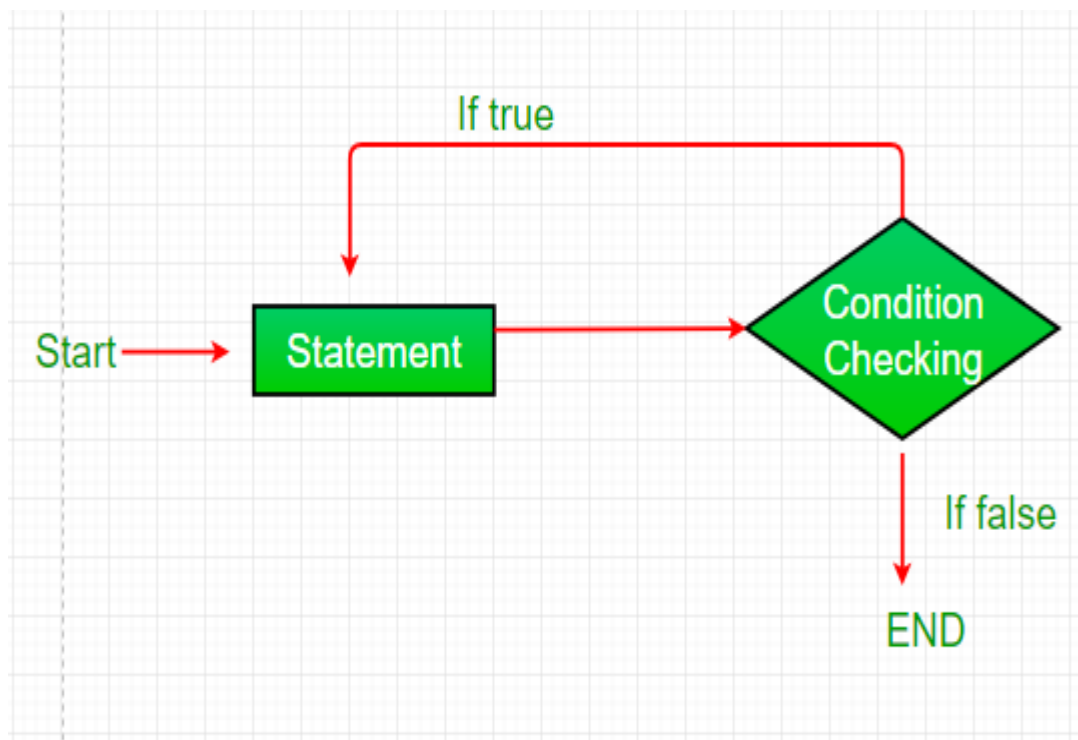
do while:

do while loop is similar to while loop with only difference that it checks for condition after executing the statements, and therefore is an example of **Exit Control Loop**.

Syntax:

```
do
{
    statements..
}
while (condition);
```

Flowchart:



- do while loop starts with the execution of the statement(s). There is no checking of any condition for the first time.
- After the execution of the statements, and update of the variable value, the condition is checked for true or false value. If it is evaluated to true, next iteration of loop starts.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.
- It is important to note that the do-while loop will execute its statements atleast once before any condition is checked, and therefore is an example of exit control loop.

```
// Java program to illustrate do-while loop
class dowhileloopDemo
{
    public static void main(String args[])
    {
        int x = 21;
        do
        {
            // The line will be printed even
            // if the condition is false
            System.out.println("Value of x:" + x);
            x++;
        }
        while (x < 20);
    }
}
```

Output:

```
Value of x: 21
```

1.10 What is Command Line Argument?

Command Line Argument is information passed to the program when you run the program. The passed information is stored as a string array in the main method. Later, you can use the command line arguments in your program.

Example While running a class **Demo**, you can specify command line arguments as

```
java Demo arg1 arg2 arg3 ...
```

Command Line Arguments in Java: Important Points

- Command Line Arguments can be used to specify configuration information while launching your application.
- There is no restriction on the number of java command line arguments. You can specify any number of arguments
- Information is passed as Strings.
- They are captured into the String args of your main method

Example:

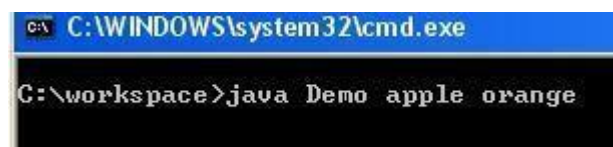
To Learn java Command Line Arguments

Step 1) Copy the following code into an editor.

```
class Demo{  
    public static void main(String b[]){  
        System.out.println("Argument one = "+b[0]);  
        System.out.println("Argument two = "+b[1]);  
    }  
}
```

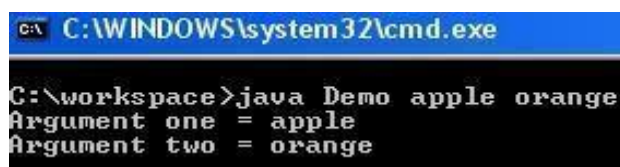
Step 2) Save & Compile the code

Step 3) Run the code as **java Demo apple orange**.



```
C:\WINDOWS\system32\cmd.exe  
C:\workspace>java Demo apple orange
```

Step 4) You must get an output as below.



```
C:\WINDOWS\system32\cmd.exe  
C:\workspace>java Demo apple orange  
Argument one = apple  
Argument two = orange
```

Program for Command Line Arguments;

```
class CommandLineExample {  
    public static void main(String[] args) {  
  
        System.out.println("Number of arguments: " + args.length);  
  
        for(int i = 0; i < args.length; i++) {  
            System.out.println("Argument " + i + ": " + args[i]);  
        }  
    }  
}
```

Output:

```
Number of arguments: 3  
Argument 0: Hello  
Argument 1: 123  
Argument 2: Java
```

1.11 Object Oriented Programming (OOPs) Concept in Java

Object-oriented programming: As the name suggests, Object-Oriented Programming or OOPs refers to languages that uses objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism etc. in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

OOPs Concepts:

[Polymorphism](#)

[Inheritance](#)

[Encapsulation](#)

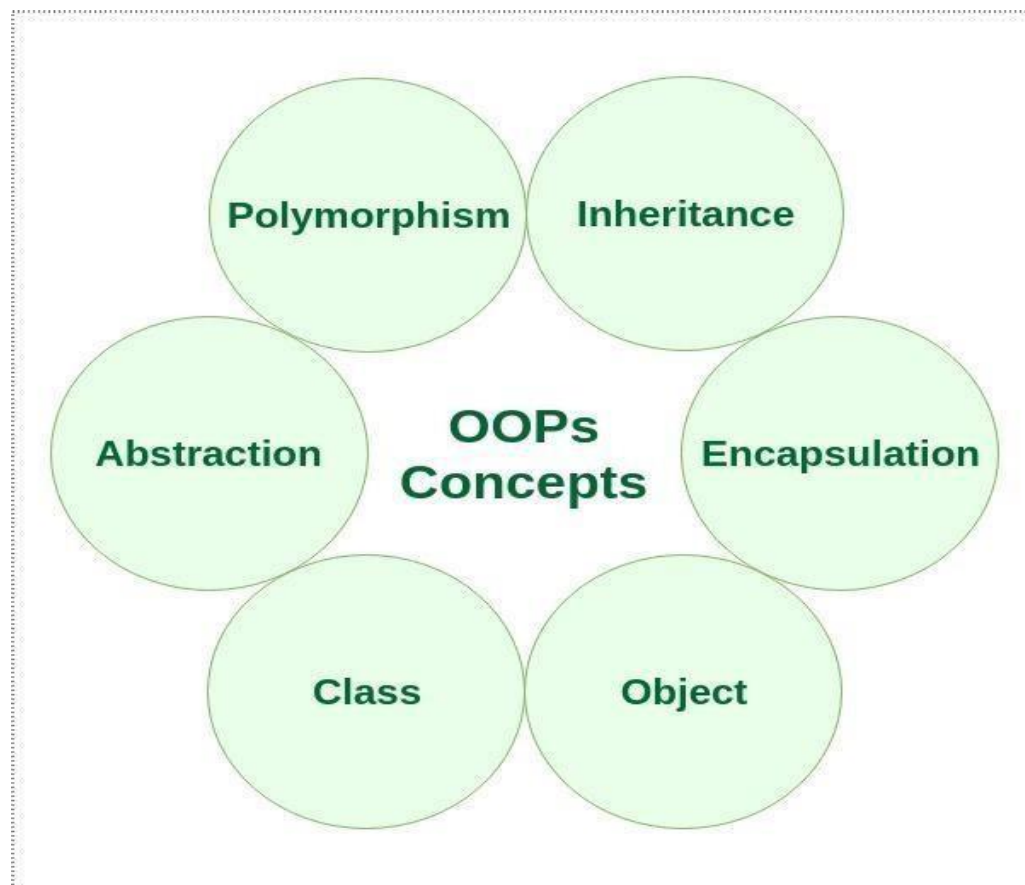
[Abstraction](#)

[Class](#)

[Object](#)

[Method](#)

[Message Passing](#)



Let us learn about the different characteristics of an Object-Oriented Programming language:

Polymorphism

Polymorphism refers to the ability of OOPs programming languages to differentiate between entities with the same name efficiently. This is done by Java with the help of the signature and declaration of these entities.

For example:

```
// Java program to demonstrate Polymorphism

// This class will contain

// 3 methods with same name,
// yet the program will
// compile & run successfully
public

class Sum {

    // Overloaded sum().
    // This sum takes two int parameters
    public int sum(int x, int y)
    {
        return (x + y);
    }

    // Overloaded sum().
    // This sum takes three int parameters
    public int sum(int x, int y, int z)
    {
        return (x + y + z);
    }

    // Overloaded sum().
    // This sum takes two double parameters
    public double sum(double x, double y)
    {
        return (x + y);
    }

    // Driver code
    public static void main(String args[])
    {
        Sum s = new Sum();
        System.out.println(s.sum(10, 20));
        System.out.println(s.sum(10, 20, 30));
        System.out.println(s.sum(10.5, 20.5));
    }
}
```

```
}  
}
```

Output:

```
30  
60  
31.0
```

Polymorphism in Java are mainly of 2 types:

- [Overloading in Java](#)
- [Overriding in Java](#)

Encapsulation:

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. Another way to think about encapsulation is, it is a protective shield that prevents the data from being accessed by the code outside this shield.

- Technically in encapsulation, the variables or data of a class is hidden from any other class and can be accessed only through any member function of own class in which they are declared.
- As in encapsulation, the data in a class is hidden from other classes, so it is also known as **data-hiding**.
- Encapsulation can be achieved by Declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables.



Abstraction

Data Abstraction is the property by virtue of which only the essential details are displayed to the user. The trivial or the non-essentials units are not displayed to the user.

Example: A car is viewed as a car rather than its individual components.

Data Abstraction may also be defined as the process of identifying only the required characteristics of an object ignoring the irrelevant details. The properties and behaviours of an object differentiate it from other objects of similar type and also help in classifying/grouping the objects.

Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of car or applying brakes will stop the car but he does not know about how on pressing the accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car. This is what abstraction is. In java, abstraction is achieved by interfaces and abstract classes. We can achieve 100% abstraction using interfaces.

Inheritance

Inheritance is an important pillar of OOP (Object Oriented Programming). It is the mechanism in java by which one class is allow to inherit the features (fields and methods) of another class.

Important terminology:

- **Super Class:** The class whose features are inherited is known as superclass(or a base class or a parent class).
- **Sub Class:** The class that inherits the other class is known as subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

The keyword used for inheritance is **extends**.

Syntax:

```
class derived-class extends base-class
{
    //methods and fields
}
```

Class:

A class is a user defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. In general, class declarations can include these components, in order:

1. **Modifiers:** A class can be public or has default access (Refer [this](#) for details).
2. **Class name:** The name should begin with a initial letter (capitalized by convention).
3. **Superclass (if any):** The name of the class's parent (superclass), if any, preceded by the keyword `extends`. A class can only extend (subclass) one parent.
4. **Interfaces (if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword `implements`. A class can implement more than one interface.
5. **Body:** The class body surrounded by braces, `{ }`.

Before you create objects in Java, you need to define a class. A class is a blueprint for the object. We can think of class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows etc. Based on these descriptions we build the house. House is the object.

Since, many houses can be made from the same description, we can create many objects from a class.

How to define a class in Java?

Here's how a class is defined in Java:

```
1.    class ClassName {  
2.        // variables  
3.        // methods  
4.    }
```

Object:

It is a basic unit of Object-Oriented Programming and represents the real-life entities. A typical Java program creates many objects, which as you know, interact by invoking methods.

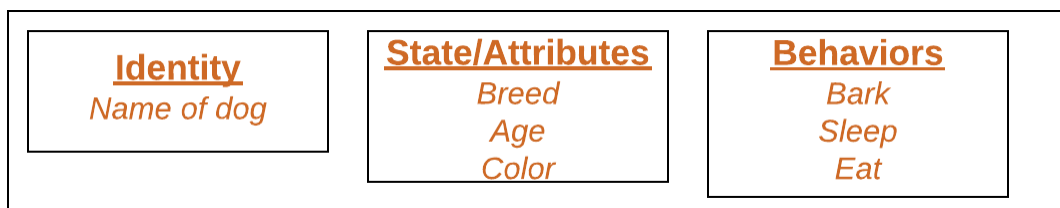
An object consists of:

1. **State:** It is represented by attributes of an object. It also reflects the properties of an object.
2. **Behavior:** It is represented by methods of an object. It also reflects the response of an object with other objects.
3. **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

Let's take few examples:

- Lamp is an object
- It can be in on or off state.
- You can turn on and turn off lamp (behavior).
- Bicycle is an object
- It has current gear, two wheels, number of gear etc. states.
- It has braking, accelerating, changing gears etc. behavior.

Example of an object: dog

**Method:**

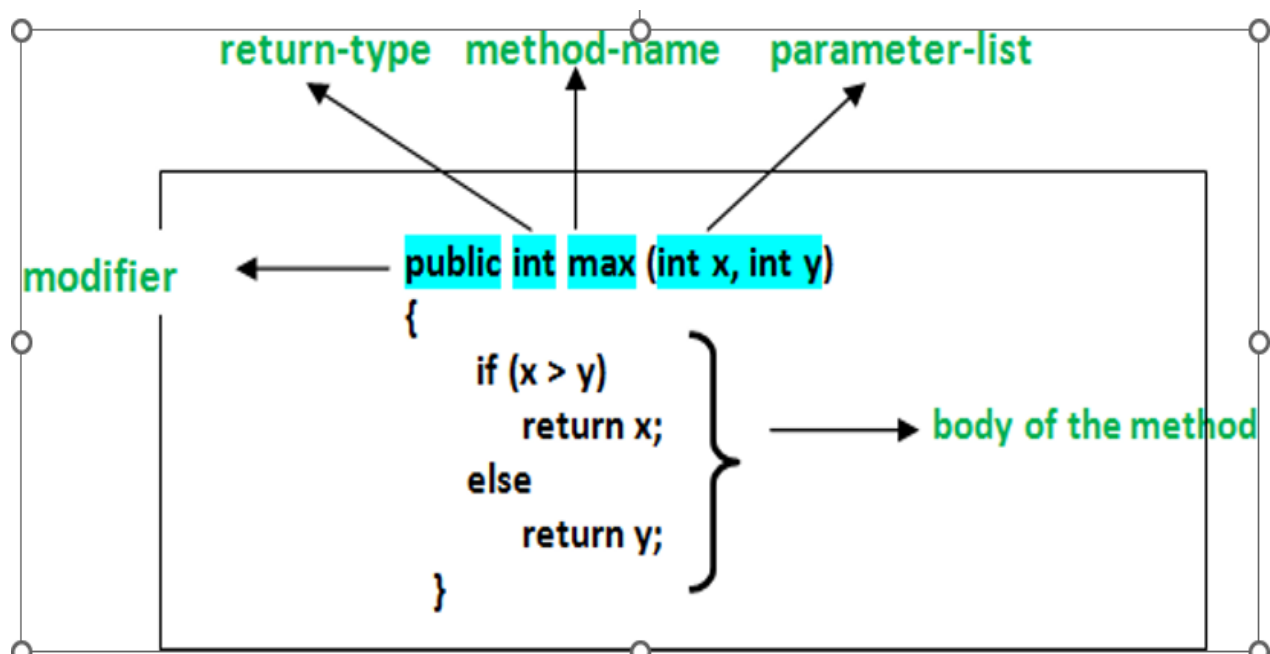
A method is a collection of statements that perform some specific task and return result to the caller. A method can perform some specific tasks without returning anything. Methods allow us to reuse the code without retyping the code. In Java, every method must be part of some class which is different from languages like C, C++ and Python. Methods are **time savers** and help us to **reuse** the code without retyping the code.

Method Declaration

In general, method declarations have six components:

- **Access Modifier:** Defines **access type** of the method i.e. from where it can be accessed in your application. In Java, there 4 types of the access specifiers.
 - **public:** accessible in all class in your application.

- **protected:** accessible within the package in which it is defined and, in its subclass,(es)(including subclasses declared outside the package)
 - **private:** accessible only within the class in which it is defined.
 - **default (declared/defined without using any modifier):** accessible within same class and package within which its class is defined.
-
- **The return type:** The data type of the value returned by the method or void if does not return a value.
 - **Method Name:** the rules for field names apply to method names as well, but the convention is a little different.
 - **Parameter list:** Comma separated list of the input parameters are defined, preceded with their data type, within the enclosed parenthesis. If there are no parameters, you must use empty parentheses ().
 - **Exception list:** The exceptions you expect by the method can throw, you can specify these exception(s).
 - **Method body:** It is enclosed between braces. The code you need to be executed to perform your intended operations.



Message Passing:

Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function and the information to be sent.

Example:

Create a class named **Lamp**.

The class should contain:

- an instance variable **isOn**
- three methods: **turnOn()**, **turnOff()**, and **displayLightStatus()**

In the **main()** method, create two objects of the Lamp class: **l1** and **l2**.

Call **l1.turnOn()** — this should set the **isOn** variable of object **l1** to **true**.

Call **l2.turnOff()** — this should set the **isOn** variable of object **l2** to **false**.

Finally:

l1.displayLightStatus(); should print “**Light on? true**” because **isOn** holds **true** for **l1**.

l2.displayLightStatus(); should print “**Light on? false**” because **isOn** holds **false** for **l2**.

When class is defined, only the specification for the object is defined; no memory or storage is allocated.

To access members defined within the class, you need to create objects. Let's create objects of Lamp class.

Here's an example:

```
1.      class Lamp {  
2.  
3.          // instance variable  
4.          private boolean isOn;  
5.  
6.          // method  
7.  
8.          public void turnOn() {  
9.              isOn = true;  
10.         }  
11.         // method  
12.         public void turnOff() {  
13.             isOn = false;  
14.         }  
15.     }
```

Here, we defined a class named Lamp.

The class has one instance variable (variable defined inside class) `isOn` and two methods `turnOn()` and `turnOff()`. These variables and methods defined within a class are called **members** of the class.

Notice two keywords, `private` and `public` in the above program. These are access modifiers which will be discussed in detail in later chapters. For now, just remember:

- The `private` keyword makes instance variables and methods private which can be accessed only from inside the same class.
- The `public` keyword makes instance variables and methods public which can be accessed from outside of the class.
-

In the above program, `isOn` variable is private whereas `turnOn()` and `turnOff()` methods are public.

If you try to access private members from outside of the class, compiler throws error.

```
class Lamp {  
    ... ..  
    void turnOn() {  
        isOn = true;  
    }  
    ... ..  
}  
  
class ClassObjectsExample {  
    public static void main(String[] args) {  
        ... ..  
        l1.turnOn();  
        ... ..  
    }  
}
```

```
1.      class Lamp {
2.          boolean isOn;
3.
4.          void turnOn() {
5.              isOn = true;
6.          }
7.
8.          void turnOff() {
9.              isOn = false;
10.         }
11.     }
12.
13.     class ClassObjectsExample {
14.     public static void main(String[] args) {
15.         Lamp l1 = new Lamp(); // create l1 object of Lamp class
16.         Lamp l2 = new Lamp(); // create l2 object of Lamp class
17.     }
18. }
```

This program creates two objects l1 and l2 of class Lamp.

How to access members?

You can access members (call methods and access instance variables) by using . operator.

For example,

```
l1.turnOn();
```

This statement calls turnOn() method inside Lamp class for l1 object.

We have mentioned word method quite a few times. You will learn about Java methods in detail in the next chapter. Here's what you need to know for now:

When you call the method using the above statement, all statements within the body of turnOn() method are executed. Then, the control of program jumps back to the statement following l1.turnOn();

Similarly, the instance variable can be accessed as:

```
l2.isOn = false;
```

It is important to note that, the private members can be accessed only from inside the class. If the code l2.isOn = false; lies within the main() method (outside of the Lamp class), compiler will show error.

Example: Java Class and Objects

```
1.      class Lamp {
2.          boolean isOn;
3.
4.          void turnOn() {
5.              isOn = true;
6.          }
7.
8.          void turnOff() {
9.              isOn = false;
10.         }
11.
12.         void displayLightStatus() {
13.
14.             System.out.println("Light on? " + isOn);
15.         }
16.     }
17.
18.
19.     class ClassObjectsExample {
20.     public static void main(String[] args) {
21.
22.         Lamp l1 = new Lamp(), l2 = new Lamp();
23.
24.         l1.turnOn();
25.         l2.turnOff();
26.
27.         l1.displayLightStatus();
28.         l2.displayLightStatus();
29.     }
30. }
```

When you run the program, the output will be:

```
Light on? true
Light on? false
```

In the above program,

- Lamp class is created.
- The class has an instance variable isOn and three methods turnOn(), turnOff() and displayLightStatus().
- Two objects l1 and l2 of Lamp class are created in the main() function.
- Here, turnOn() method is called using l1 object: l1.turnOn();

- This method sets isOn instance variable of l1 object to true.
- And, turnOff() method is called using l2 object: l2.turnOff();
- This method sets isOff instance variable of l2 object to false.
- Finally, l1.displayLightStatus(); statement displays Light on? true because isOn variable holds true for l1 object.
- And, l2.displayLightStatus(); statement displays Light on? false because isOn variable holds false for l2 object

Note, variables defined within a class are called **instance variable** for a reason.

When an object is initialized, it's called an instance. Each instance contains its own copy of these variables. For example, isOn variable for objects l1 and l2 are different

1.10 Method Overloading in Java

Methods are used in Java to describe the behavior of an object. Methods are a collection of statements that are group together to operate. In Java, it is possible to create methods that have the same name, but different argument lists in various definitions, i.e., method overloading is possible in Java, which is one of the unique features of Object-Oriented Programming (OOP). In this chapter, we will learn about how method overloading is written and how it helps us within a Java program.

What is Method Overloading in Java?

If a class of a Java program has a plural number of methods, and all of them have the same name but different parameters (with a change in type or number of arguments), and programmers can use them to perform a similar form of functions, then it is known as method overloading. If programmers what to perform one operation, having the same name, the method increases the readability if the program.

Let us take an example where you can perform multiplication of given numbers, but there can be any number of arguments a user can put to multiply, i.e., from the user point of view the user can multiply two numbers or three numbers or four numbers or so on. If you write the method such as `multi(int, int)` with two parameters for multiplying two values, `multi(int, int, int)`, with three parameters for multiplying three values and so on. A programmer does method overloading to figure out the program quickly and efficiently. Method Overloading is applied in a program when objects are required to perform similar tasks but different input parameters. Every time an object calls a method, Java matches up to the method name first and then the number and type of parameters to decide what definitions to execute. This process of assigning multiple tasks to the same method is known as *Polymorphism*. There are different ways to overload a method in Java. They are:

- Based on the number of parameters: In this case, the entire overloading concept depends on the number of parameters that are placed within the parenthesis of the method.
- Based on the data type of the parameter: With the arrangement of data type, within the parameter, overloading of the method can take place.
- Based on the sequence of data types in parameters: The method overloading also depends on the ordering of data types of parameters within the method.

To create overloaded methods, programmers need to develop several different method definitions in the class, all have the same name, but the parameters list is different.

Syntax:

```
public class Student {  
  
    public void add(int i, int j) {  
  
        ....  
    }  
  
    public void add(int i) {  
  
        ....  
    }  
}
```

The static polymorphism is also called compile-time binding or early binding. Static binding happens at compile time, and Method overloading is an example of static binding where binding of the method call to its definition occurs at Compile time.

Program for Method Overloading in Java

Example:

```
class Multiply {  
  
    void mul(int a, int b) {  
  
        System.out.println("Sum of two="+(a * b));  
    }  
  
    void mul(int a, int b, int c) {
```

```
System.out.println("Sum of three="+(a * b * c));

}

}

public class Polymorphism{

public static void main(String args[]){

Multiply m =new Multiply();

    m.mul(6,10);

    m.mul(10,6,5);

}

}
```

Output:

Sum of two=60

Sum of three=300

Why Method Overloading is not possible if the return type of method is changed?

In Java, method overloading is not possible by changing the return type of the method because there may arise some ambiguity. Let's see how ambiguity may occur:

Example:

```
class overloadRetType {

int sum(int g,int h){

System.out.println(g + h);

}

double sum(int g,int h){

System.out.println(g + h);

}

public static void main(String args[]){
```

```
        overloadRetType ob =new overloadRetType();

        int result = ob.sum(20,20);

        //The above line will produce a compile Time Error ....

    }

}
```

The above program will generate a compile-time error. Here, Java cannot determine which sum() method to call and hence creates an error if you want to overload like this by using the return type. Overloading methods offer no specific benefit to the JVM, but it is useful to the program to have several ways do the same things but with different parameters.

Here are some other examples to show Method Overloading:

Program to Demonstrate Method Overloading Based on the Number of Parameters

Example:

```
class DispOvrload

{

    public void show(char ch)

    {

        System.out.println ("You have typed the letter: "+ch);

    }

    public void show(char ch, char ch1)

    {

        System.out.println("You have typed the letter: "+ch+", and "+ ch1);

    }

}

class Main

{
```

```
public static void main (String args[])  
{  
    DispOvrload o1 =new DispOvrload();  
    o1.show('G');  
    o1.show('S','J');  
}  
}
```

Output:

You have typed the letter: G

You have typed the letter: S, and J

Program to Demonstrate Method Overloading Based on the Sequence of Data Type in the Parameters

Example:

```
class DispOvrload  
{  
    public void show(char ch,int numb)  
    {  
        System.out.println ("The 'show method' is defined for the first time.");  
    }  
    public void show(int numb ,char ch)  
    {  
        System.out.println ("The 'show method' is defined for the second time.");  
    }  
}  
  
class Main
```

```
{  
  
public static void main (String args[])  
  
{  
  
DispOvrload o1 =new DispOvrload();  
  
    o1.show('G',62);  
  
    o1.show(46,'S');  
  
}  
  
}
```

Output:

The 'show method' is defined for the first time.

The 'show method' is defined for the second time.

Advantages of Method Overloading

- It is used to perform a task efficiently with smartness in programming.
- It increases the readability of the program.
- The Method overloading allows methods that perform proximately related functions to be accessed using a common name with slight variation in argument number or types.
- They can also be implemented on constructors allowing different ways to initialize objects of a class.

Disadvantages of Method Overloading

- It's esoteric. Not very easy for the beginner to opt this programming technique and go with it.
- It requires more significant effort spent on designing the architecture (i.e., the arguments' type and number) to up front, at least if programmers' want to avoid massive code from rewriting.

1.11 Constructor in java

Constructor is a block of code that initializes the newly created object. A constructor resembles an instance method in java but it's not a method as it doesn't have a return type. In short constructor and method are different (More on this at the end of this guide). People often refer constructor as special type of method in Java.

Constructor has same name as the class and looks like this in a java code.

```
public class MyClass{  
//This is the constructor  
MyClass(){  
}  
..  
}
```

Note that the constructor name matches with the class name and it doesn't have a return type.

How does a constructor work

To understand the working of constructor, let's take an example. Let's say we have a class MyClass.

When we create the object of MyClass like this:

```
MyClass obj =newMyClass()
```

The new keyword here creates the object of class MyClass and invokes the constructor to initialize this newly created object.

You might feel a bit confused at this point because we haven't discussed initialization examples yet.

So let's look at a simple constructor example to understand how initialization works in Java.

In this program, we create an object obj of the class Hello and then print its instance variable name.

The output is: BeginnersBook.com

This happens because the value "BeginnersBook.com" was passed to the constructor when the object was created, and the constructor assigned this value to the instance variable name. This clearly shows that the constructor gets invoked automatically at the time of object creation.

In this example, this keyword is used inside the constructor. The keyword this refers to the current object, which in our case is the object obj. We will study this keyword in detail in the next tutorial.


```
public class Hello{  
    String name;  
    //Constructor  
    Hello(){  
        this.name ="BeginnersBook.com";  
    }  
    public static void main(String[] args){  
        Hello obj =new Hello();  
        System.out.println(obj.name);  
    }  
}
```

Output:

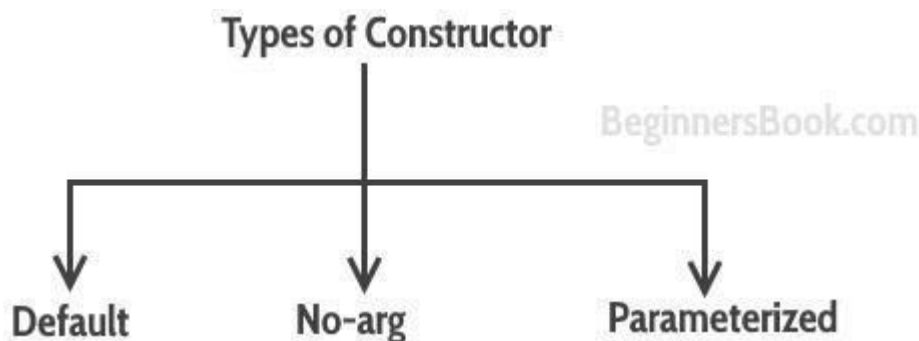
BeginnersBook.com

```
public class MyClass{  
    // Constructor  
    MyClass(){  
        System.out.println("BeginnersBook.com");  
    }  
    public static void main(String args[]){  
        MyClass obj = new MyClass();  
        ...  
    }  
}
```

New keyword creates the object of MyClass & invokes the constructor to initialize the created object.

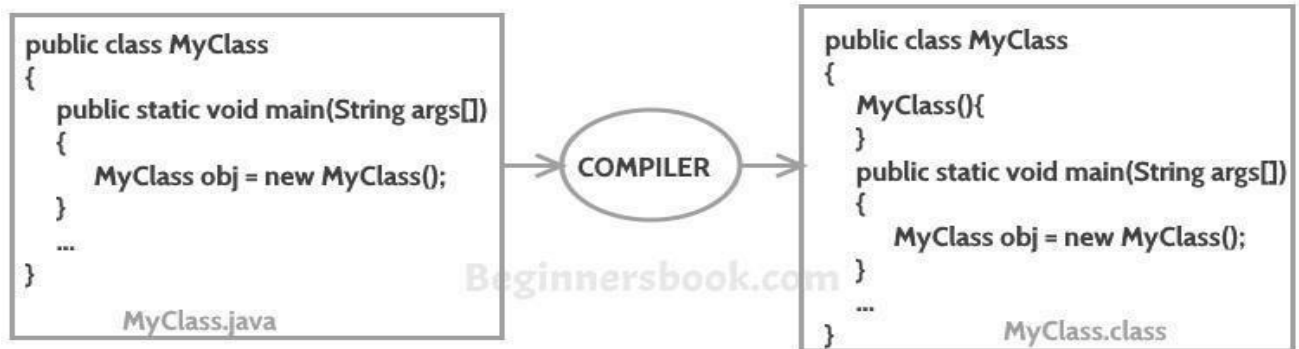
Types of Constructors

There are three types of constructors: Default, No-arg constructor and Parameterized.



Default constructor

If you do not implement any constructor in your class, Java compiler inserts a default constructor into your code on your behalf. This constructor is known as default constructor. You would not find it in your source code (the java file) as it would be inserted into the code during compilation and exists in .class file. This process is shown in the diagram below:



If you implement any constructor then you no longer receive a default constructor from Java compiler.

no-arg constructor:

Constructor with no arguments is known as no-arg constructor. The signature is same as default constructor; however, body can have any code unlike default constructor where the body of the constructor is empty.

Although you may see some people claim that that default and no-arg constructor is same but in fact they are not, even if you write **public Demo() { }** in your class `Demo` it cannot be called default constructor since you have written the code of it.

Example: no-arg constructor

```
class Demo
{
    public Demo()
    {
        System.out.println("This is a no argument constructor");
    }
    public static void main(String args[]) {
        new Demo();
    }
}
```

Output:

This is a no argument constructor

Parameterized constructor

Constructor with arguments(or you can say parameters) is known as Parameterized constructor.

Example 1: parameterized constructor

In this example we have a parameterized constructor with two parameters id and name. While creating the objects obj1 and obj2 I have passed two arguments so that this constructor gets invoked after creation of obj1 and obj2.

Output:

```
public class Employee{

int empId;
String empName;

//parameterized constructor with two parameters
Employee(int id,String name){
this.empId = id;
this.empName =
name;
}
void info(){
System.out.println("Id: "+empId+" Name: "+empName);
}

public static void main(String args[]){
    Employee obj1 =new Employee(10245,"Chaitanya");
    Employee obj2 =new Employee(92232,"Negan");
    obj1.info();
    obj2.info();
}
```

Output:

```
Id:10245Name:Chaitanya
Id:92232Name:Negan
```

Example2: parameterized constructor

In this example, we have two constructors, a default constructor and a parameterized constructor. When we do not pass any parameter while creating the object using new keyword then default constructor is invoked, however when you pass a parameter then parameterized constructor that matches with the passed parameters list gets invoked.

```
class Example2
{
    private int var;
    //default constructor
    public Example2()
    {
        this.var=10;
    }
    //parameterized constructor
    public Example2(int num)
    {
        this.var= num;
    }

    public int getValue()
    {
        return var;
    }
    public static void main(String args[])
    {
        Example2 obj =new Example2();
        Example2 obj2 =new Example2(100);
        System.out.println("var is: "+obj.getValue());
        System.out.println("var is: "+obj2.getValue());
    }
}
```

Output:

```
varis:10
varis:100
```

What if you implement only parameterized constructor in class

```
classExample3
{
privateintvar;
publicExample3(int num)
{
var=num;
}
publicint getValue()
{
returnvar;
}
publicstaticvoid main(String args[])
{
Example3 myobj =newExample3();
System.out.println("value of var is: "+myobj.getValue());
}
}
```

Output:

It will throw a compilation error. The reason is, the statement `Example3 myobj = new Example3()` is invoking a default constructor which we don't have in our program. when you don't implement any constructor in your class, compiler inserts the default constructor into your code, however when you implement any constructor (in above example I have implemented parameterized constructor with int parameter), then you don't receive the default constructor by compiler into your code.

If we remove the parameterized constructor from the above code then the program would run fine, because then compiler would insert the default constructor into your code

1.14 Static method in java

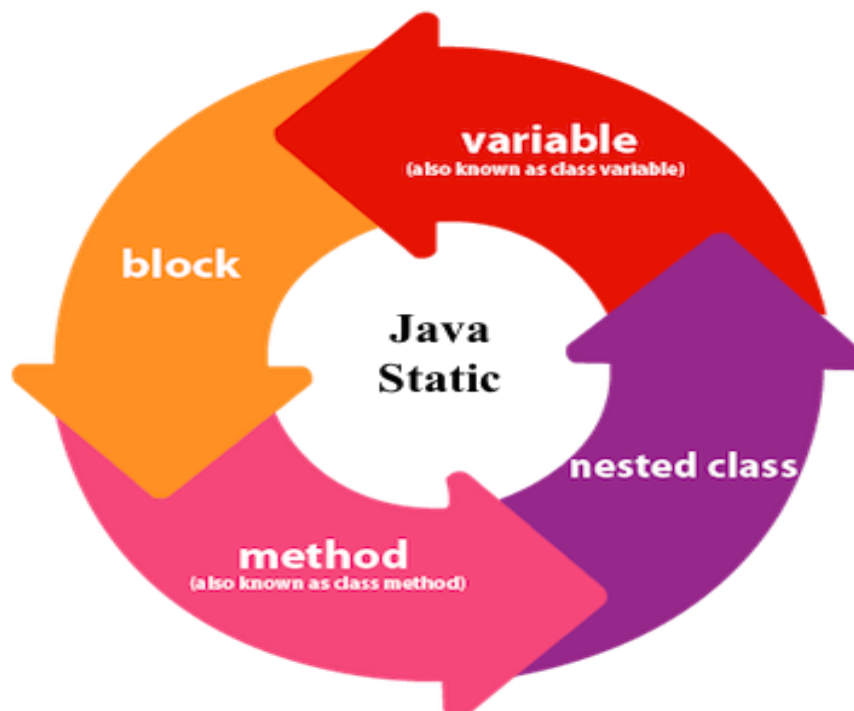
Java static keyword

1. [Static variable](#)
2. [Program of the counter without static variable](#)
3. [Program of the counter with static variable](#)
4. [Static method](#)
5. [Restrictions for the static method](#)
6. [Why is the main method static?](#)
7. [Static block](#)
8. [Can we execute a program without main method?](#)

The **static keyword** in [Java](#) is used for memory management mainly. We can apply static keyword with [variables](#), methods, blocks and [nested classes](#). The static keyword belongs to the class than an instance of the class.

The static can be:

1. Variable (also known as a class variable)
2. Method (also known as a class method)
3. Block
4. Nested class



Java static variable

If you declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.

Advantages of static variable

It makes your program **memory efficient** (i.e., it saves memory).

Understanding the problem without static variable

```
1.    class Student{  
2.        int rollno;  
3.        String name;  
4.        String college="ITS";  
5.    }
```

Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created. All students have its unique rollno and name, so instance data member is good in such case. Here, "college" refers to the common property of all [objects](#). If we make it static, this field will get the memory only once.

Java static property is shared to all objects.

Example of static variable

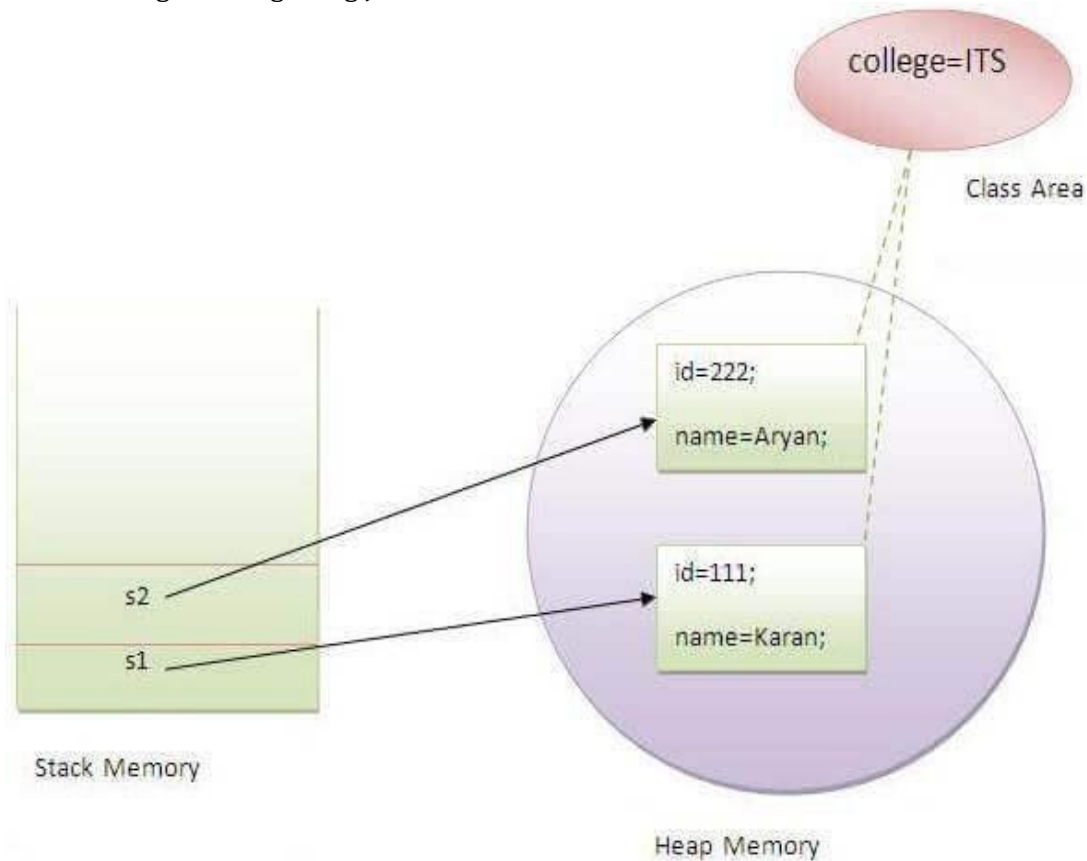
```
//Java Program to demonstrate the use of static variable  
class Student{  
    int rollno;//instance variable  
    String name;  
    static String college ="ITS";//static variable  
    //constructor  
    Student(int r, String n){  
        rollno = r;  
        name = n;  
    }  
    //method to display the values
```

```
void display () {System.out.println(rollno+" "+name+" "+college);}
}
//Test class to show the values of objects
public class TestStaticVariable1 {
public static void main(String args[]){
Student s1 = new Student(111,"Karan");

Student s2 = new Student(222,"Aryan");
//we can change the college of all objects by the single line of code
//Student.college="BBDIT";
s1.display();
s2.display();
}
}
```

Output:

```
111 Karan ITS
222 Aryan ITS
```

Program of the counter without static variable

In this example, we have created an instance variable named `count` which is incremented in the constructor. Since instance variable gets the memory at the time of object creation, each object will have the copy of the instance variable. If it is incremented, it won't reflect other objects. So each object will have the value 1 in the `count` variable.

```
//Java Program to demonstrate the use of an instance variable  
//which get memory each time when we create an object of the class.
```

```
class Counter{  
int count=0;//will get memory each time when the instance is created  
  
Counter(){  
count++;//incrementing value  
System.out.println(count);  
  
}  
  
public static void main(String args[]){  
//Creating objects  
Counter c1=new Counter();  
Counter c2=new Counter();  
Counter c3=new Counter();  
}  
}
```

Output:

```
1  
1  
1
```

Program of counter by static variable

As we have mentioned above, static variable will get the memory only once, if any object changes the value of the static variable, it will retain its value.

```
//Java Program to illustrate the use of static variable which  
//is shared with all objects.  
class Counter2{  
static int count=0;//will get memory only once and retain its value  
  
Counter2(){  
count++;//incrementing the value of static variable  
System.out.println(count);  
}  
  
public static void main(String args[]){  
//creating objects  
Counter2 c1=new  
Counter2(); Counter2  
c2=new Counter2();
```

```
Counter2      c3=new  
Counter2();  
}  
}
```

Output:

```
1  
2  
3
```

Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Example of static method

//Java Program to demonstrate the use of a static method.

```
class Student{ int  
rollno; String  
name;  
static String college = "ITS";  
//static method to change the value of static variable  
static void change(){ college  
= "BBDIT";  
}  
//constructor to initialize the variable Student(int  
r, String n){  
rollno = r;  
name = n;  
}  
//method to display values  
void display(){System.out.println(rollno+" "+name+" "+college);}  
}
```

//Test class to create and display the values of object

```
public class TestStaticMethod{  
    public static void main(String args[]){  
        Student.change();//calling change method  
        //creating objects  
        Student s1 = new Student(111,"Karan");  
        Student s2 = new Student(222,"Aryan");  
        Student s3 = new Student(333,"Sonoo");  
        //calling display method  
        s1.display();  
        s2.display();  
        s3.display();  
    } }  

```

Output:

```
111 Karan BBDIT  
222 Aryan BBDIT  
333 Sonoo BBDIT
```

Another example of a static method that performs a normal calculation

//Java Program to get the cube of a given number using the static method

```
class Calculate{  
    static int cube(int x){  
        return x*x*x;  
    }  
  
    public static void main(String args[]){  
        int result=Calculate.cube(5);  
        System.out.println(result);  
    }  
}
```

Output:125

Restrictions for the static method

There are two main restrictions for the static method. They are:

1. The static method cannot use non static data member or call non-static method directly.
2. this and super cannot be used in static context.

```
class A{
    int a=40;//non static

    public static void main(String args[]){
        System.out.println(a);
    }
}
```

Output:
Compile Time Error

Q) Why is the Java main method static?

Ans) It is because the object is not required to call a static method. If it were a non-static method, [JVM](#) creates an object first then call main() method that will lead the problem of extra memory allocation.

Java static block

- Is used to initialize the static data member.
- It is executed before the main method at the time of classloading.

Example of static block

```
1.    class A2{
2.        static{System.out.println("static block is invoked");}
3.        public static void main(String args[]){
4.            System.out.println("Hello main");
5.        }
6.    }
```

Output:
static block is invoked
Hello main

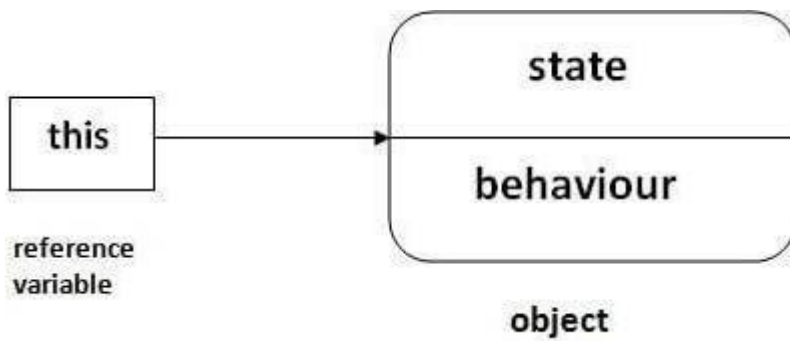
Q) Can we execute a program without main() method?

Ans) No, one of the ways was the static block, but it was possible till JDK 1.6. Since JDK 1.7, it is not possible to execute a Java class without the [main method](#).

```
1.    class A3{
2.        static{
3.            System.out.println("static block is invoked");
4.            System.exit(0);
5.        }
6.    }
```

Output:
static block is invoked

1.15 this keyword in java



There can be a lot of usage of **java this keyword**. In java, this is a **reference variable** that refers to the current object.

Usage of java this keyword

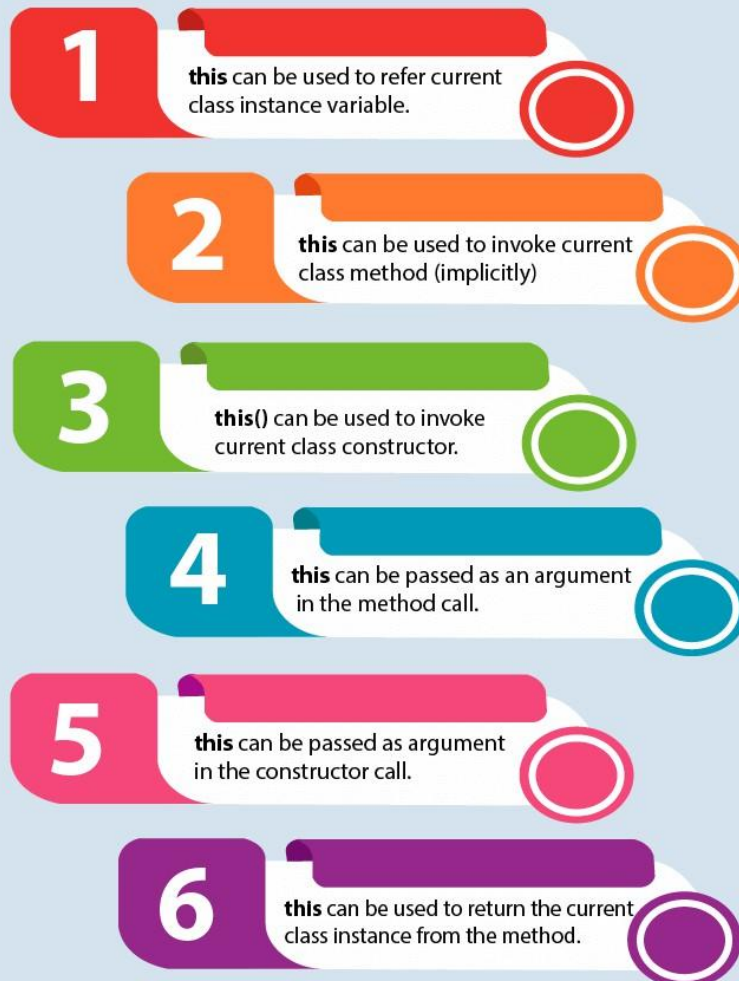
Here is given the 6 usages of java this keyword.

- i. this can be used to refer current class instance variable.
- ii. this can be used to invoke current class method (implicitly)
- iii. this() can be used to invoke current class constructor.
- iv. this can be passed as an argument in the method call.
- v. this can be passed as argument in the constructor call.
- vi. this can be used to return the current class instance from the method.

Suggestion: If you are beginner to java, lookup only three usage of this keyword.

Usage of java this keyword

There can be a lot of usage of java this keyword. In java, this is a reference variable that refers to the current object.



1) **this**: to refer current class instance variable

The **this** keyword can be used to refer current class instance variable. If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

Understanding the problem without this keyword

Let's understand the problem if we don't use this keyword by the example given below:

1. **class** Student{
2. **int** rollno;

```
3.      String name;

4.      float fee;
5.      Student(int rollno,String name,float fee){
6.      rollno=rollno;
7.      name=name;
8.      fee=fee;
9.      }
10.     void display(){System.out.println(rollno+" "+name+" "+fee);}
11.     }
12.     class TestThis1 {
13.     public static void main(String args[]){
14.     Student s1=new Student(111,"ankit",5000f);
15.     Student s2=new Student(112,"sumit",6000f);
16.     s1.display();
17.     s2.display();
18.     }}
```

Output:

```
0 null 0.0
0 null 0.0
```

In the above example, parameters (formal arguments) and instance variables are same. So, we are using this keyword to distinguish local variable and instance variable.

Solution of the above problem by this keyword

```
1.      class Student{
2.      int rollno;
3.      String name;
4.      float fee;
5.      Student(int rollno,String name,float fee){
6.      this.rollno=rollno;
7.      this.name=name;
8.      this.fee=fee;
9.      }
10.     void display(){System.out.println(rollno+" "+name+" "+fee);}
11.     }
12.
13.     class TestThis2 {
14.     public static void main(String args[]){
15.     Student s1=new Student(111,"ankit",5000f);
16.     Student s2=new Student(112,"sumit",6000f);
17.     s1.display();
18.     s2.display();
19.     }}
```


Output:

```
111 ankit 5000  
112 sumit 6000
```

If local variables(formal arguments) and instance variables are different, there is no need to use this keyword like in the following program:

Model Question Bank

1. Define the Java Development Kit (JDK).
2. List the primary Java buzzwords and their meanings.
3. What is bytecode in Java?
4. Define JVM and explain its role.
5. List different Java primitive data types.
6. State the syntax rules for declaring arrays in Java.
7. What are control statements in Java? Name any three.
8. Define method overloading with a simple example.
9. What is the purpose of a constructor in Java?
10. What is the function of the this keyword?
11. Define the Java Development Kit (JDK).
12. List the primary Java buzzwords and their meanings.
13. What is bytecode in Java?
14. Define JVM and explain its role.
15. List different Java primitive data types.
16. State the syntax rules for declaring arrays in Java.
17. What are control statements in Java? Name any three.
18. Define method overloading with a simple example.
19. What is the purpose of a constructor in Java?
20. Explain the relationship between JVM, JDK, and JRE.
21. Describe how Java achieves platform independence.
22. Explain how arrays are stored and accessed in Java.
23. Illustrate the use of command-line arguments in Java with an example.

